COMSATS University Islamabad Wah Campus



Department of Computer Science

Internship Report (Week No. 4)

Submitted By :

Registration No : FA23-BAI-004

Name : Atif Ali Shah

Program : BSAI

Organization : Pakistan Ordnance Factory (POF) Wah Cantt (Rawalpindi)

Department : CO - 4 (IT)

Section : Database Administration (DBA)

Site Supervisor Name : Sir Babar Hussain **Signature :** _____

Submitted To :

Faculty Supervisor Name : Sir Ikram Ul Haq

Submission Date : 31 August 2026 (Monday)

Table of Contents

Chapter 6: Boutique Management System

Practical No. 8: Design and Development of Boutique Management System

Table of Contents

6.1 Project Introduction
6.2 Project Objectives
6.3 Project Requirements
6.4 System Modules
6.5 Entity Relationship Diagram (ERD)
6.6 Database Normalization
6.7 Schema/User Creation in TOAD
6.8 SQL Developer Connection
6.9 Table Creation
6.10 Constraints Implementation
6.11 Sequences Creation
6.12 Sample Data Insertion
6.13 Order and Payment Management

Chapter 6: Boutique Management System

Practical No. 7: Design and Development of Boutique Management System

6.1 Project Introduction

The **Boutique Management System** is a database-based project developed to manage the basic operations of a boutique business.

The system is designed using **Oracle Database 12c**. It stores information related to customers, employees, products, categories, suppliers, orders, order details, and payments.

The database is designed according to the principles of a relational database system. Primary keys, foreign keys, unique constraints, check constraints, and other database objects are used to maintain data integrity.

The database will be developed under a dedicated **BOUTIQUE database user**, which will first be created using **TOAD for Oracle**. The same user will then be connected through **Oracle SQL Developer** for database development.

The completed database will later be used in **Week No. 5 for TOAD database administration**, including monitoring, maintenance, performance tuning, backup and recovery, and security.

6.2 Project Objectives

The main objectives of the Boutique Management System are:

- To design a relational database for a boutique business.
- To create a normalized database structure.
- To manage customer information.
- To manage employee information.
- To manage products and categories.
- To maintain supplier information.
- To manage customer orders.
- To maintain order details.
- To record customer payments.
- To implement primary and foreign key constraints.
- To use sequences for generating unique IDs.
- To insert and manage sample business data.
- To develop useful SQL queries.
- To verify the integrity and functionality of the database.
- To prepare the database for administration through TOAD.

6.3 Project Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Sufficient storage space
- Network connection where required

Software Requirements

- Oracle Database 12c
- Oracle SQL Developer
- TOAD for Oracle 12.1
- Windows operating system for SQL Developer and TOAD

6.4 System Modules

The Boutique Management System consists of the following major modules:

Module	Purpose
Customer Management	Stores customer information
Employee Management	Stores employee information
Category Management	Manages product categories
Product Management	Stores product information
Supplier Management	Stores supplier information
Order Management	Manages customer orders
Order Details	Stores products included in orders
Payment Management	Records customer payments
SQL Queries	Retrieves useful business information
Database Verification	Checks database objects and data

6.5 Entity Relationship Diagram (ERD)

The database will consist of the following main entities:

- **CUSTOMERS**
- **EMPLOYEES**
- **CATEGORIES**
- **SUPPLIERS**
- **PRODUCTS**
- **ORDERS**
- **ORDER_DETAILS**
- **PAYMENTS**

Main Relationships

Relationship Explanation

CUSTOMERS → ORDERS

One customer can place many orders, therefore there is a **one-to-many relationship**.

EMPLOYEES → ORDERS

One employee can handle multiple orders.

CATEGORIES → PRODUCTS

One category can contain multiple products.

SUPPLIERS → PRODUCTS

One supplier can supply multiple products.

ORDERS → ORDER_DETAILS

One order can contain multiple order detail records.

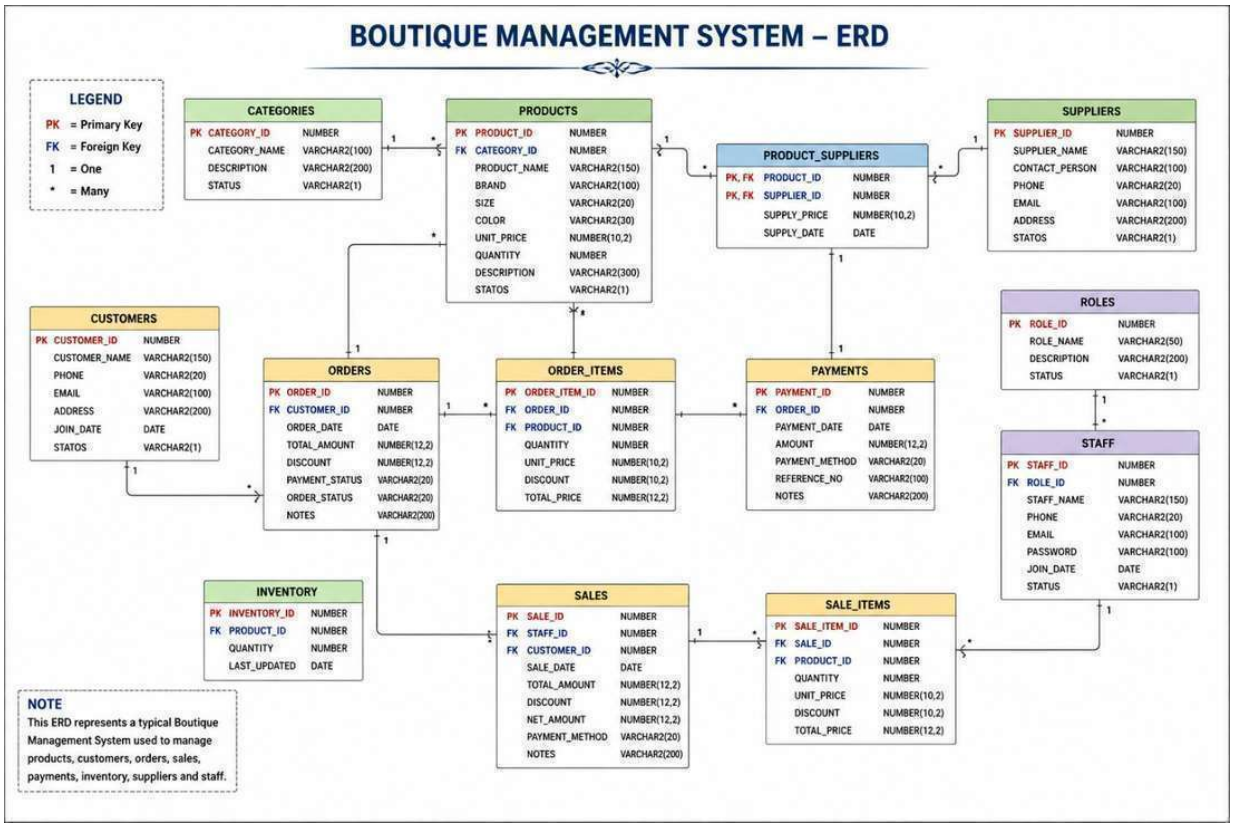
PRODUCTS → ORDER_DETAILS

One product can appear in multiple order details.

ORDERS → PAYMENTS

An order can have payment records associated with it.

Figure 6.1: Entity Relationship Diagram of Boutique Management System



6.6 Database Normalization

Normalization is the process of organizing data into related tables to reduce data redundancy and improve data integrity.

The Boutique Management System will follow the basic principles of **First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF).**

First Normal Form (1NF)

Each field contains a single atomic value and repeating groups are avoided.

Second Normal Form (2NF)

All non-key attributes depend on the complete primary key.

Third Normal Form (3NF)

Non-key attributes depend only on the primary key and not on other non-key attributes

Example

Instead of storing all information in one large table:

Customer

Product

Category

Supplier

Order

Payment

the information is separated into related tables:

CUSTOMERS

EMPLOYEES

CATEGORIES

SUPPLIERS

PRODUCTS

ORDERS

ORDER_DETAILS

PAYMENTS

This reduces duplicate data and makes the database easier to maintain.

6.7 Schema/User Creation in TOAD

The Boutique Management System will use a dedicated Oracle database user named:

BOUTIQUE

The user will be created through **TOAD for Oracle**, as instructed by the instructor.

Objective

To create a dedicated database user/schema for the Boutique Management System.

Procedure

1. Open **TOAD for Oracle**.
2. Connect to Oracle Database 12c using an administrative account.
3. Open the **Database/User Management** section.
4. Create a new database user.
5. Enter the username:

BOUTIQUE

6. Set a suitable password.
7. Assign the required privileges for database development.
8. Create/save the user.
9. Verify that the BOUTIQUE user has been created successfully.

Figure 6.3: Creating BOUTIQUE User in TOAD

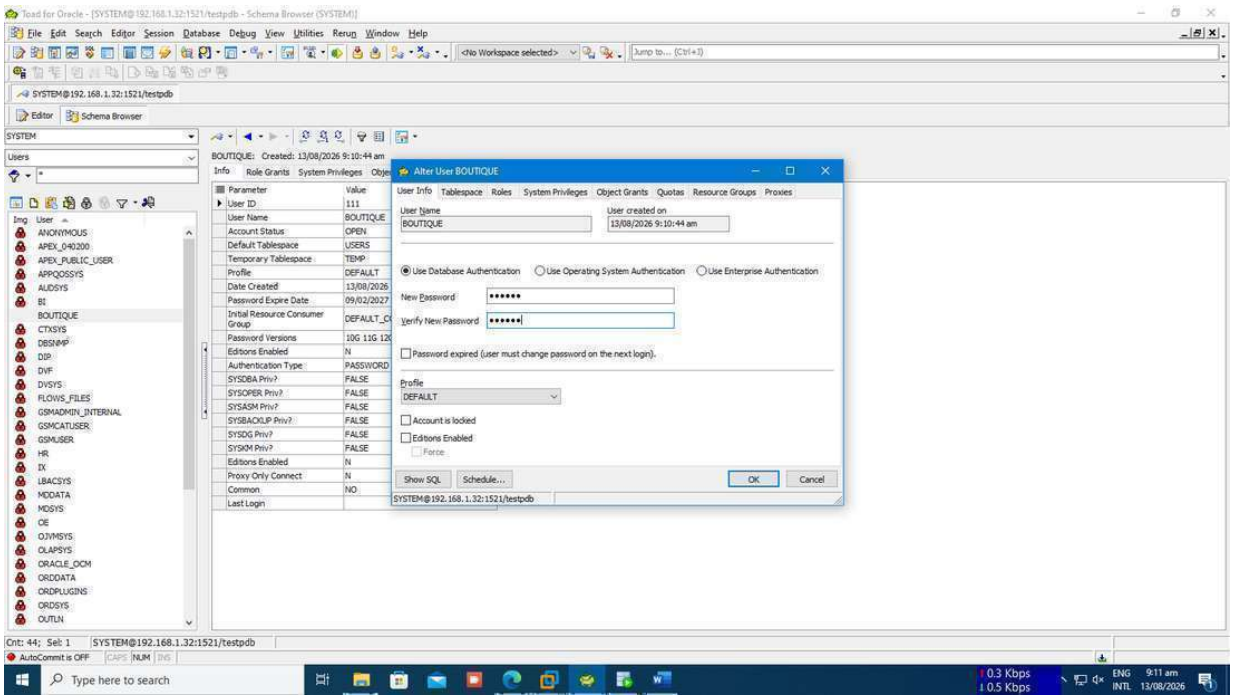
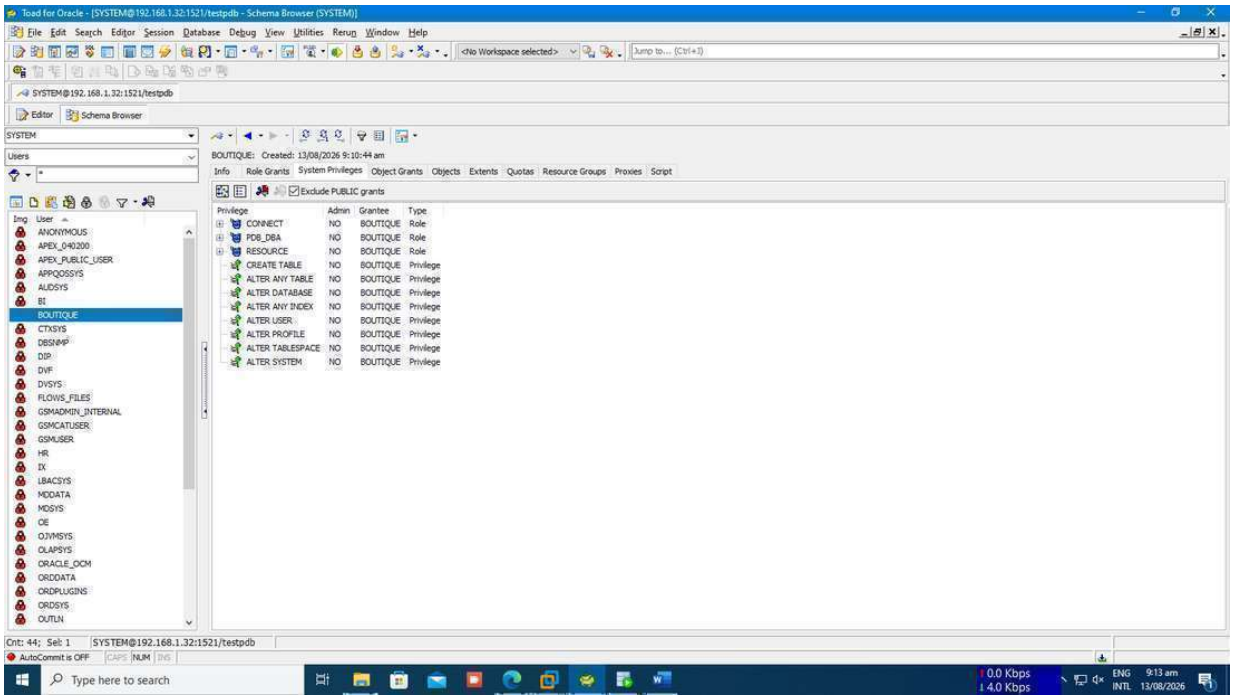


Figure 6.4: Successful BOUTIQUE User Creation



Note: Hum BOUTIQUE user ke exact privileges tumhare TOAD screen par available options dekh kar set karenge. Is stage par main unnecessary privileges assume nahi kar raha.

6.8 SQL Developer Connection

After creating the BOUTIQUE user in TOAD, the same user will be used to connect to Oracle Database through SQL Developer.

Connection Details

Username: BOUTIQUE

Password: [Password created in TOAD]

Hostname: [Oracle Database Host]

Port: 1521

Service Name: [pdborcl.virtual.pk]

Procedure

1. Open Oracle SQL Developer.
2. Click **New Connection**.
3. Enter the username created in TOAD.
4. Enter the same password.
5. Enter the Oracle database host.
6. Enter port 1521.
7. Enter the correct Oracle service name.
8. Click **Test**.
9. If the status is successful, click **Connect**.

Expected result:

Status: Success

Figure 6.5: BOUTIQUE Connection in SQL Developer

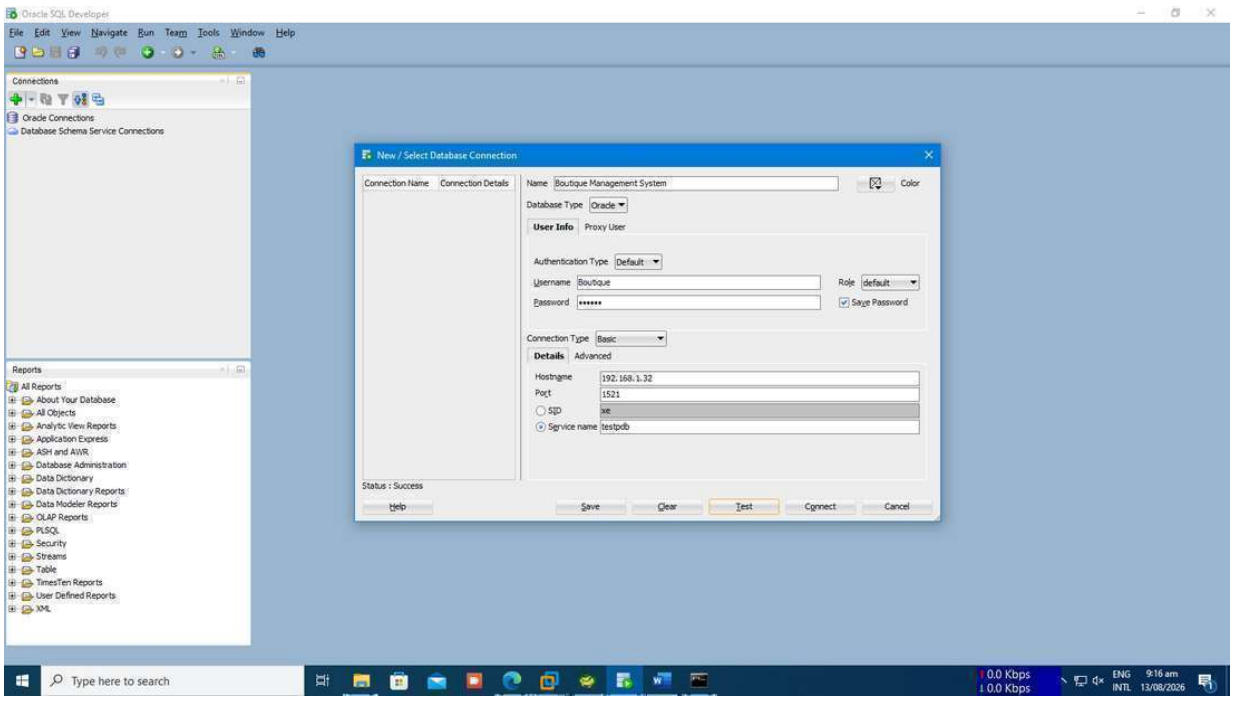
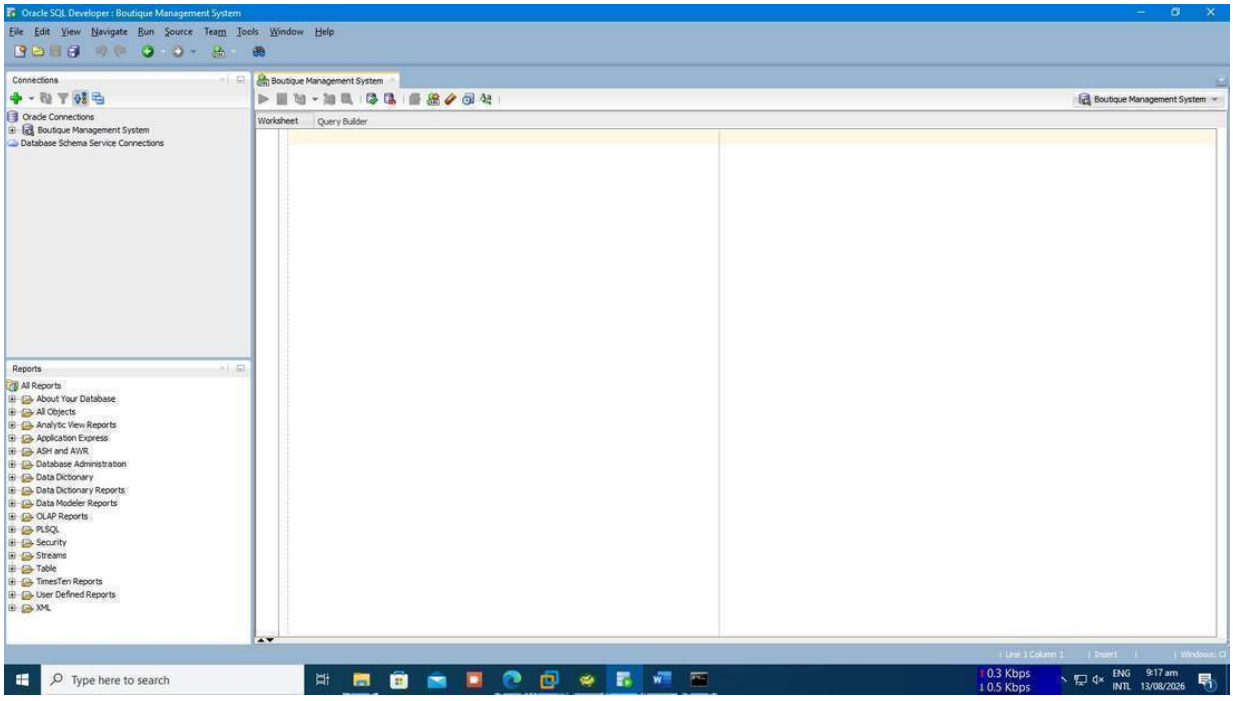


Figure 6.6: Successful SQL Developer Connection



6.9 Table Creation

Objective

The objective of this step is to create the main relational tables required for the Boutique Management System under the BOUTIQUE schema.

The tables will be created in a specific order so that foreign-key relationships can be implemented without errors.

Tables to be Created

1. CUSTOMERS
2. EMPLOYEES
3. CATEGORIES
4. SUPPLIERS
5. PRODUCTS
6. ORDERS
7. ORDER_DETAILS
8. PAYMENTS

Step 1: Verify Current User

SQL Developer mein **BOUTIQUE connection** open karo aur ye command run karo:

SELECT USER FROM DUAL;

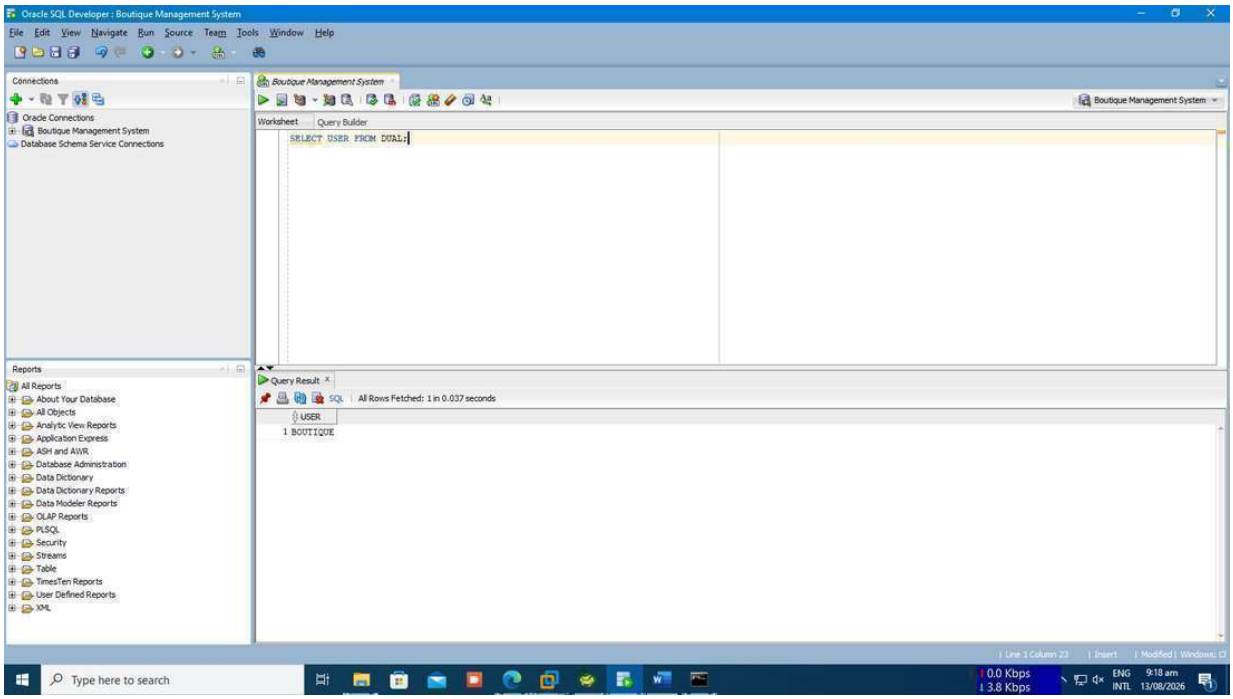
Expected Output

BOUTIQUE

Explanation

The USER function is used to verify the currently connected Oracle database user. It ensures that the project tables are being created under the BOUTIQUE schema rather than another user.

Figure 6.7: Verification of BOUTIQUE Database User



Step 2: Create CUSTOMERS Table

Ab ye command run karo:

```
CREATE TABLE customers (
   customer_id NUMBER(6),
   customer_name VARCHAR2(100) NOT NULL,
   phone VARCHAR2(20),
   email VARCHAR2(100),
   address VARCHAR2(200),
   registration_date DATE DEFAULT SYSDATE,
   CONSTRAINT pk_customers PRIMARY KEY (customer_id)
);
```

Expected Output

Table CUSTOMERS created.

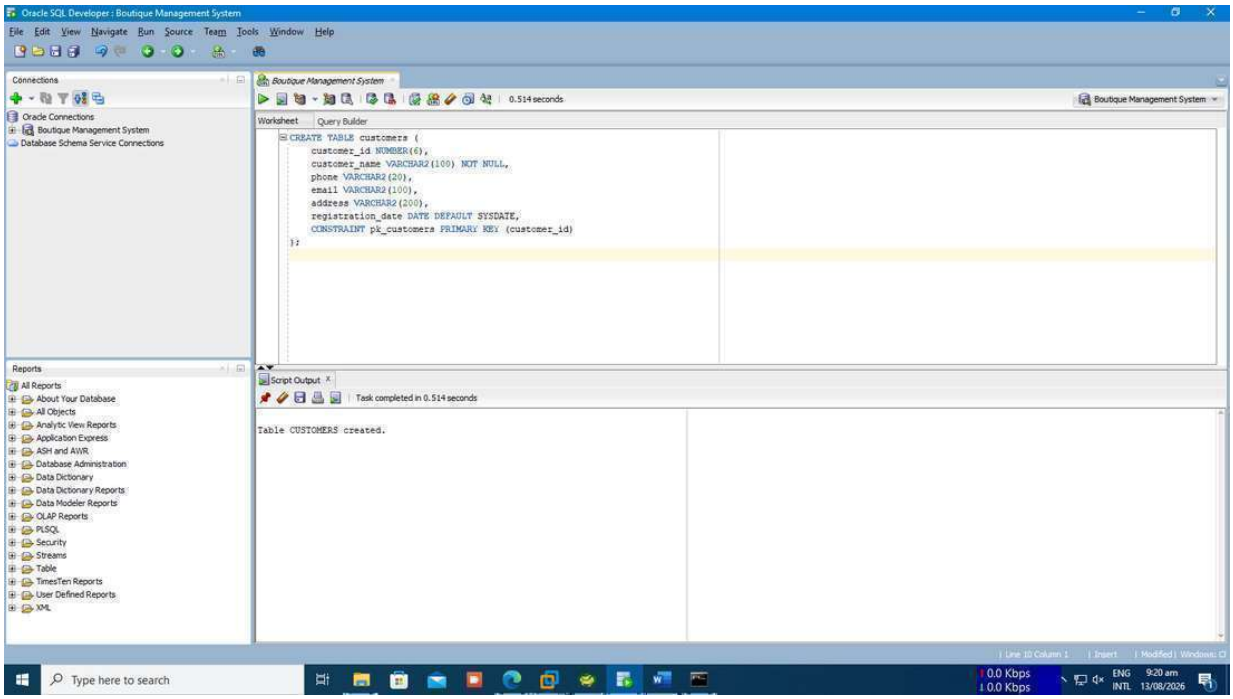
Explanation

The CUSTOMERS table stores information about boutique customers.

Column	Purpose
CUSTOMER_ID	Unique customer identification
CUSTOMER_NAME	Customer name
PHONE	Customer contact number
EMAIL	Customer email
ADDRESS	Customer address
REGISTRATION_DATE	Customer registration date

The CUSTOMER_ID column is defined as the **Primary Key**, while CUSTOMER_NAME is defined as NOT NULL.

Figure 6.8: CUSTOMERS Table Creation



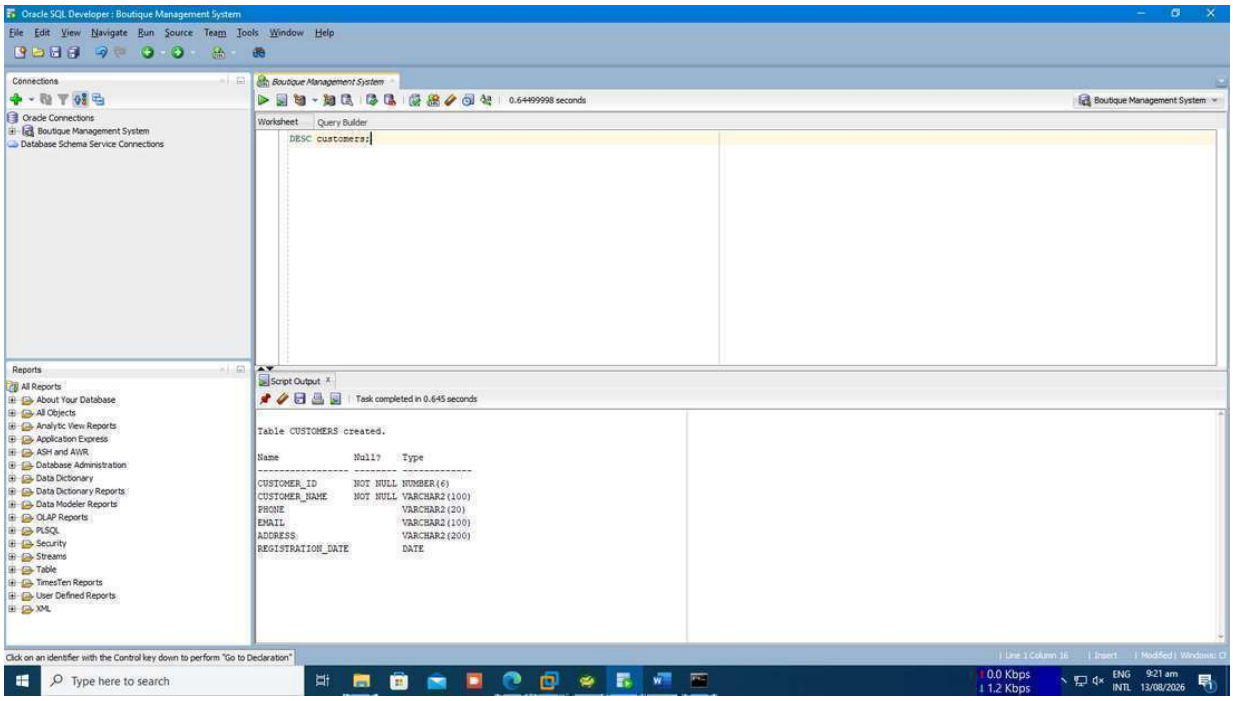
Step 3: Verify CUSTOMERS Table

Run:

DESC customers;

Expected output will display the table structure and its columns.

Figure 6.8: CUSTOMERS Table Structure



Step 4: Create EMPLOYEES Table

Run:

```
CREATE TABLE employees (
   employee_id NUMBER(6),
   employee_name VARCHAR2(100) NOT NULL,
   phone VARCHAR2(20),
   email VARCHAR2(100),
   job_title VARCHAR2(50),
   hire_date DATE DEFAULT SYSDATE,
   salary NUMBER(10,2),
   CONSTRAINT pk_employees PRIMARY KEY (employee_id)
);
```

Explanation

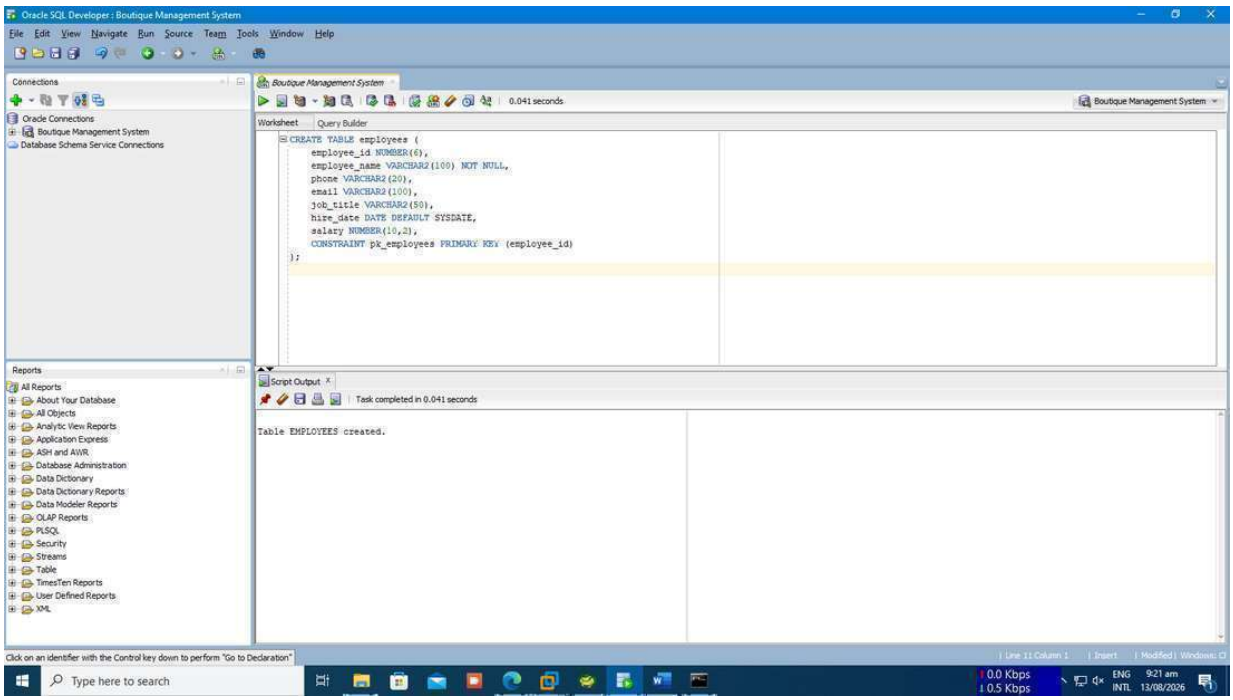
The EMPLOYEES table stores information about employees working in the boutique.

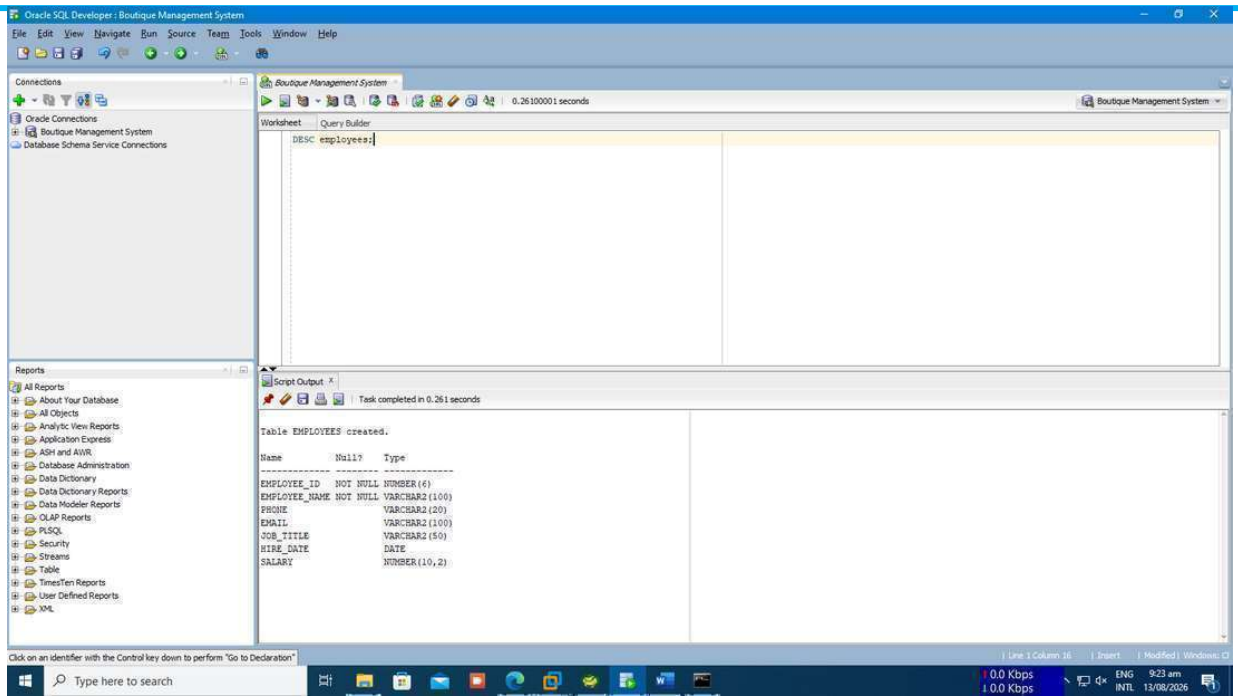
The EMPLOYEE_ID is the primary key and uniquely identifies each employee.

Verification

```
DESC employees;
```

Figure 6.9: EMPLOYEES Table Structure





Step 5: Create CATEGORIES Table

Run:

```
CREATE TABLE categories (
   category_id NUMBER(6),
   category_name VARCHAR2(100) NOT NULL,
   description VARCHAR2(200),
   CONSTRAINT pk_categories PRIMARY KEY (category_id),
   CONSTRAINT uq_category_name UNIQUE (category_name)
);
```

Explanation

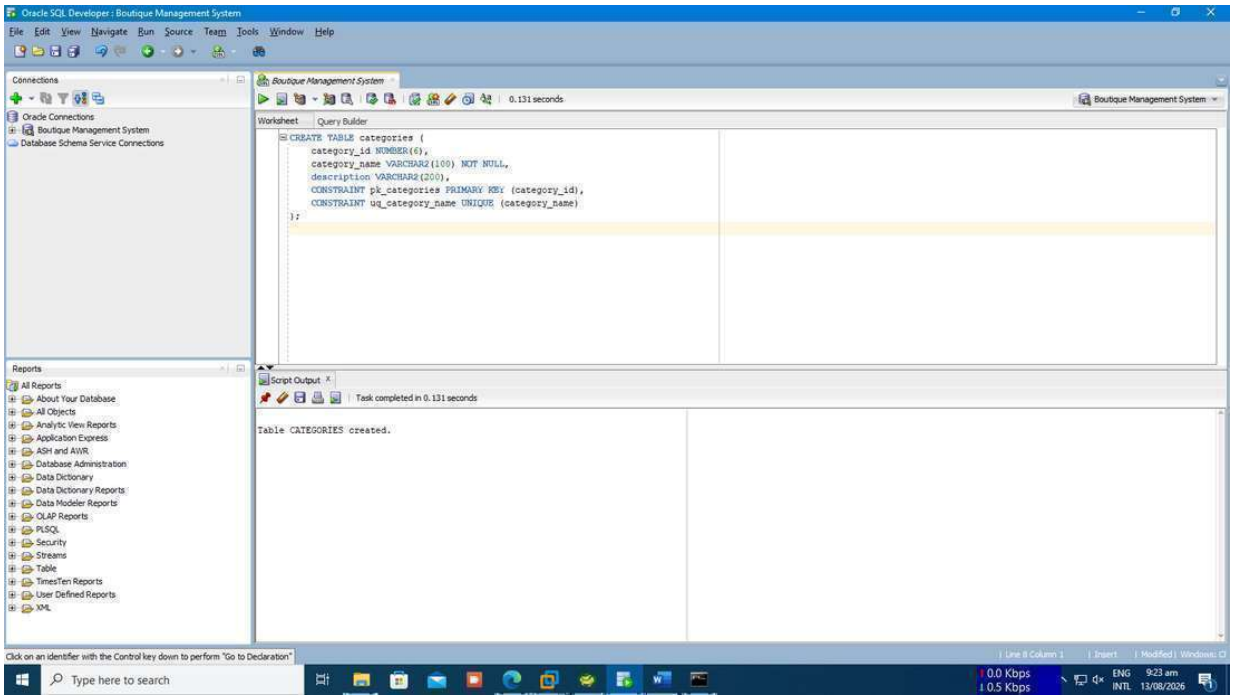
The CATEGORIES table stores product categories such as Dresses, Suits, Accessories, and Shawls.

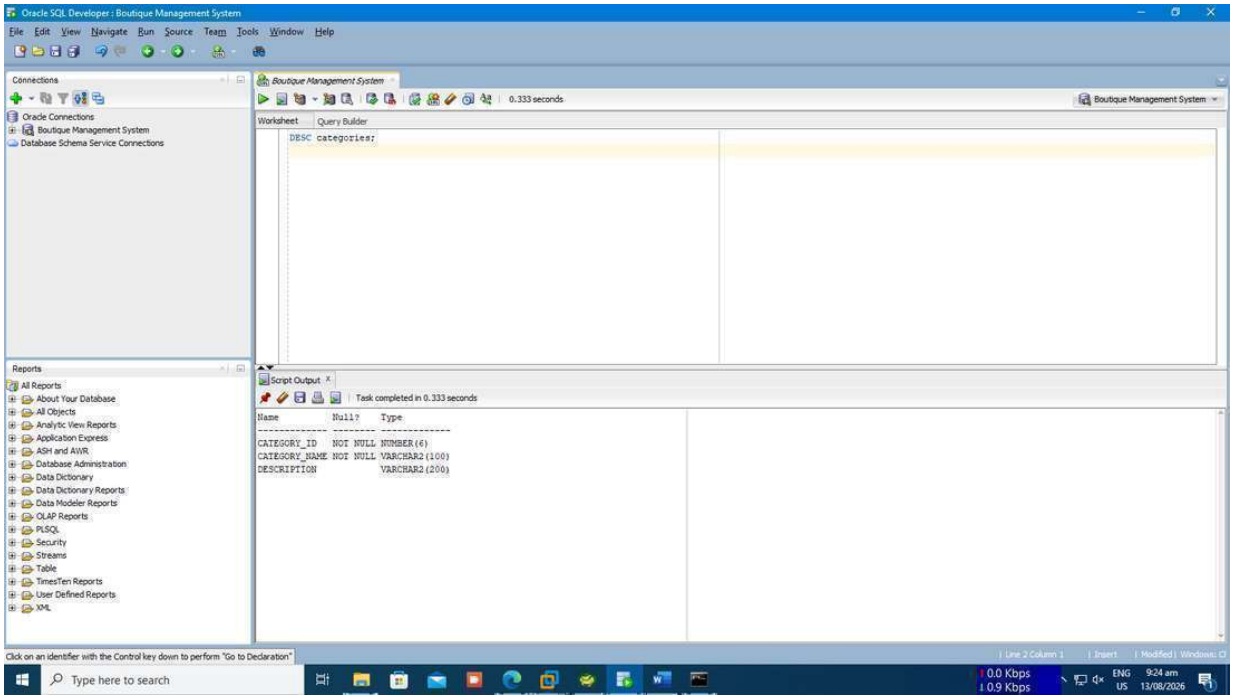
The CATEGORY_ID is the primary key, while CATEGORY_NAME has a unique constraint to prevent duplicate category names.

Verification

DESC categories;

Figure 6.10: CATEGORIES Table Structure





Step 6: Create SUPPLIERS Table

Run:

```
CREATE TABLE suppliers (
   supplier_id NUMBER(6),
   supplier_name VARCHAR2(100) NOT NULL,
   phone VARCHAR2(20),
   email VARCHAR2(100),
   address VARCHAR2(200),
   CONSTRAINT pk_suppliers PRIMARY KEY (supplier_id)
);
```

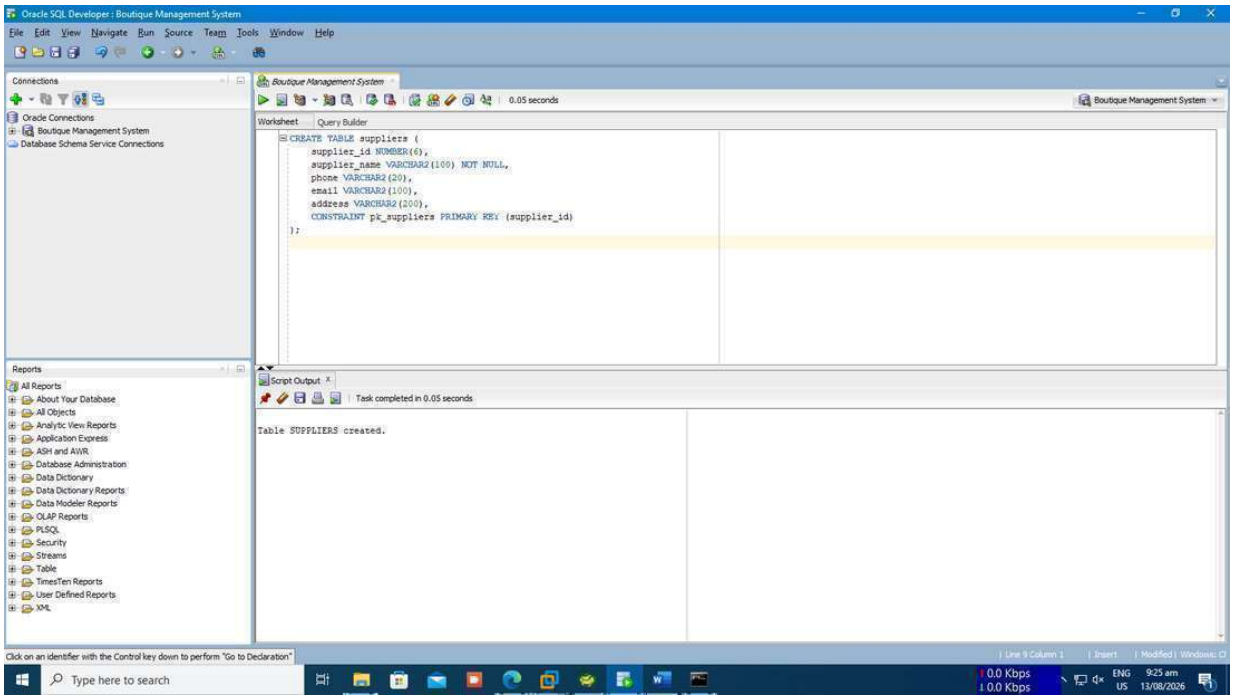
Explanation

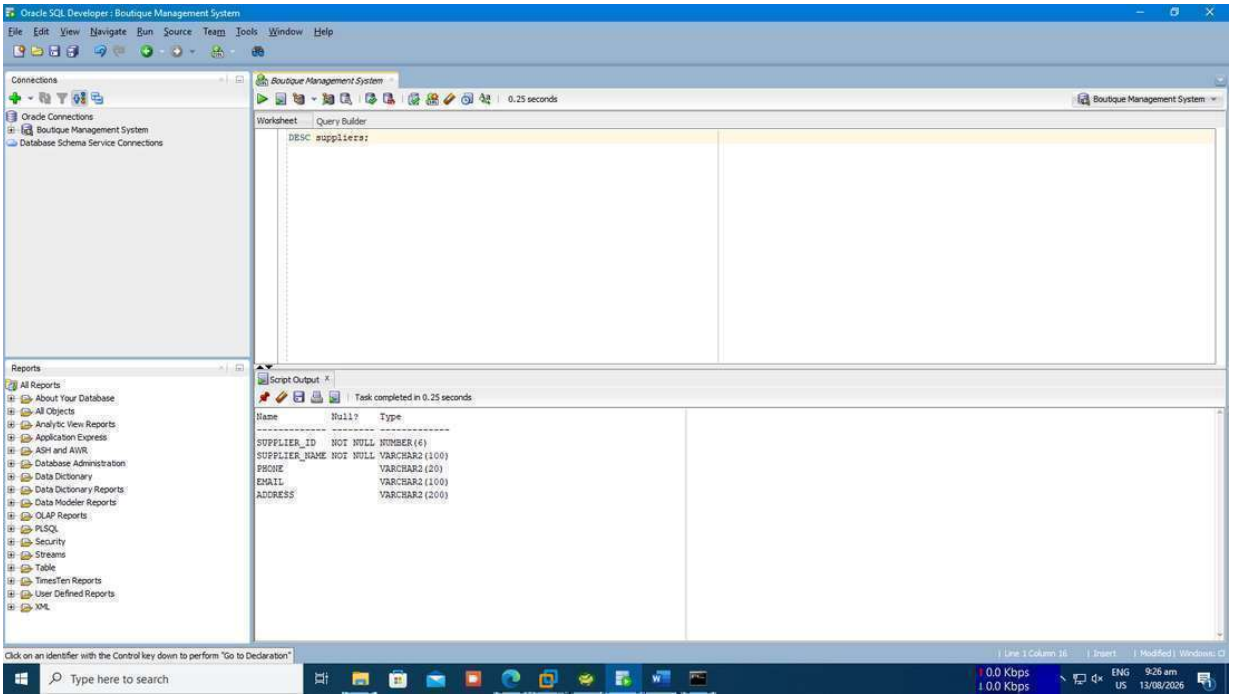
The SUPPLIERS table stores information about suppliers who provide products to the boutique.

Verification

DESC suppliers;

Figure 6.11: SUPPLIERS Table Structure





Step 7: Create PRODUCTS Table

Ab ye important table hai kyun ke is mein **Category aur Supplier ke foreign keys** aayengi.

```
CREATE TABLE products (
   product_id NUMBER(6),
   product_name VARCHAR2(100) NOT NULL,
   category_id NUMBER(6) NOT NULL,
   supplier_id NUMBER(6),
   price NUMBER(10,2) NOT NULL,
   stock_quantity NUMBER(8) DEFAULT 0,
   size_info VARCHAR2(30),
   color VARCHAR2(30),
   CONSTRAINT pk_products PRIMARY KEY (product_id),
   CONSTRAINT fk_product_category
      FOREIGN KEY (category_id)
      REFERENCES categories(category_id),
   CONSTRAINT fk_product_supplier
      FOREIGN KEY (supplier_id)
      REFERENCES suppliers(supplier_id),
   CONSTRAINT chk_product_price
      CHECK (price > 0),
   CONSTRAINT chk_product_stock
      CHECK (stock_quantity >= 0)
);
```

Explanation

The PRODUCTS table stores the products available in the boutique.

It contains foreign keys:

CATEGORY_ID → CATEGORIES

SUPPLIER_ID → SUPPLIERS

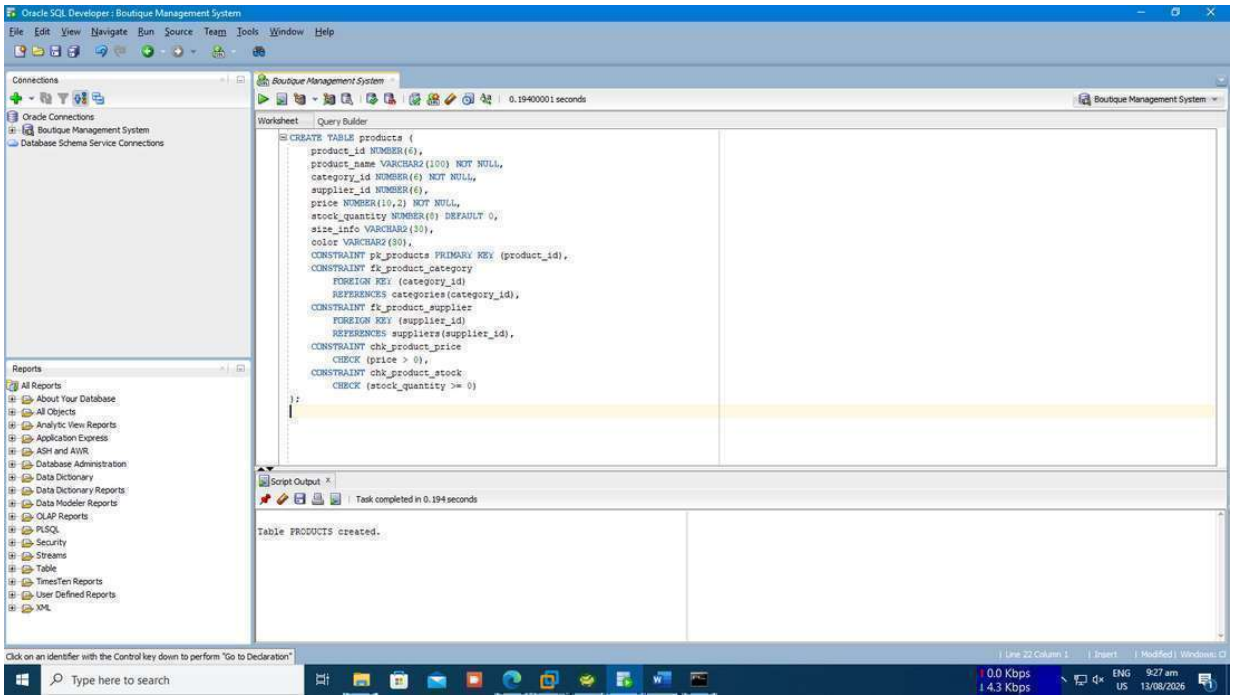
It also contains check constraints to ensure that:

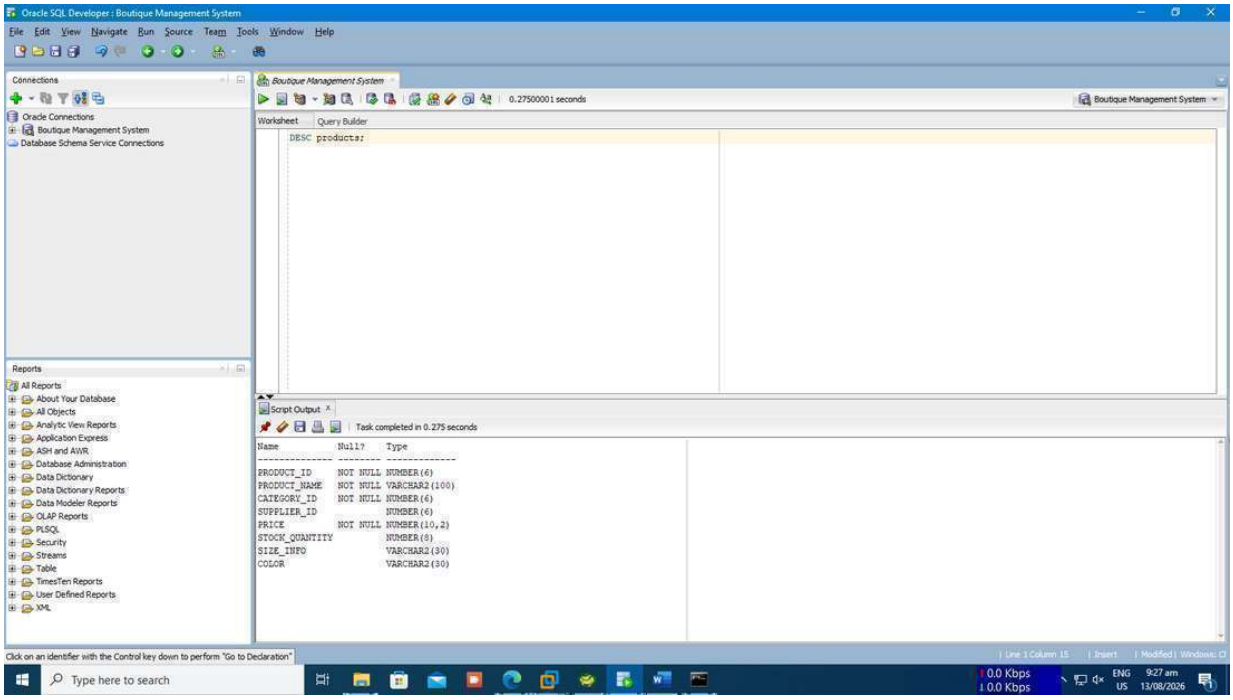
- Product price is greater than zero.
- Stock quantity is not negative.

Verification

```
DESC products;
```

Figure 6.12: PRODUCTS Table Structure





Step 8: Create ORDERS Table

```
CREATE TABLE orders (
   order_id NUMBER(6),
   customer_id NUMBER(6) NOT NULL,
   employee_id NUMBER(6),
   order_date DATE DEFAULT SYSDATE,
   order_status VARCHAR2(20) DEFAULT 'PENDING',
   total_amount NUMBER(12,2) DEFAULT 0,
   CONSTRAINT pk_orders PRIMARY KEY (order_id),
   CONSTRAINT fk_order_customer
      FOREIGN KEY (customer_id)
      REFERENCES customers(customer_id),
   CONSTRAINT fk_order_employee
      FOREIGN KEY (employee_id)
      REFERENCES employees(employee_id),
   CONSTRAINT chk_order_amount
      CHECK (total_amount >= 0)
);
```

Explanation

The ORDERS table stores customer orders.

Relationships:

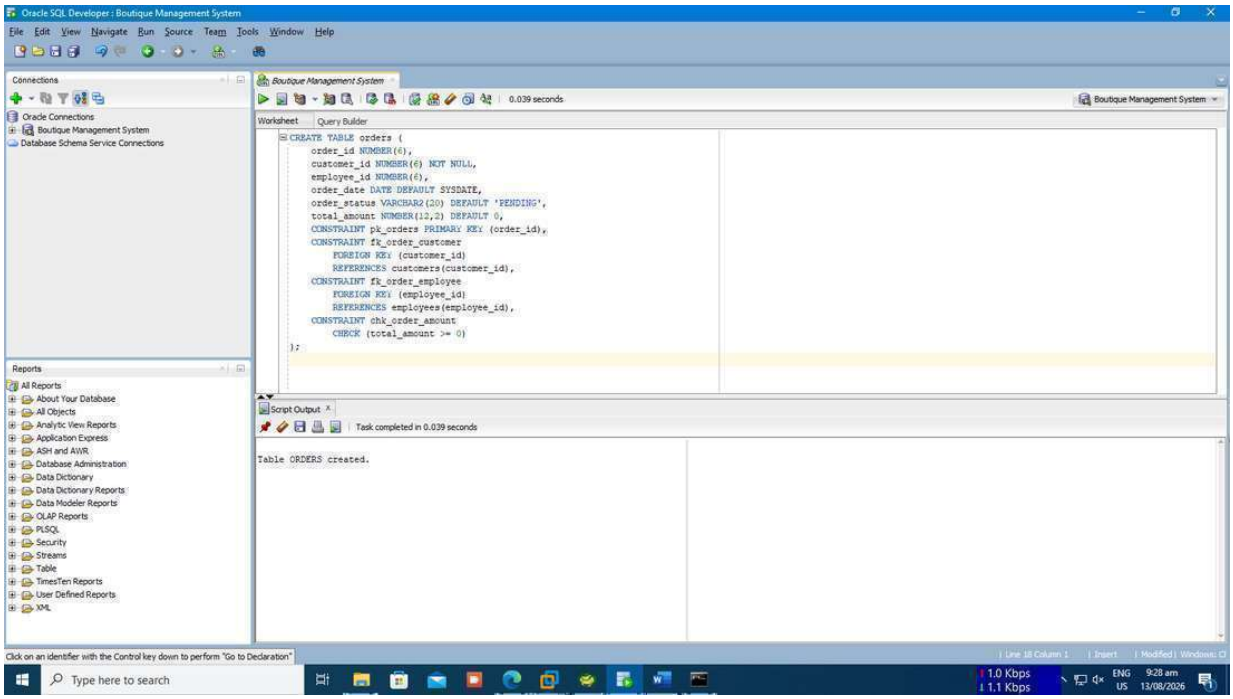
CUSTOMERS → ORDERS

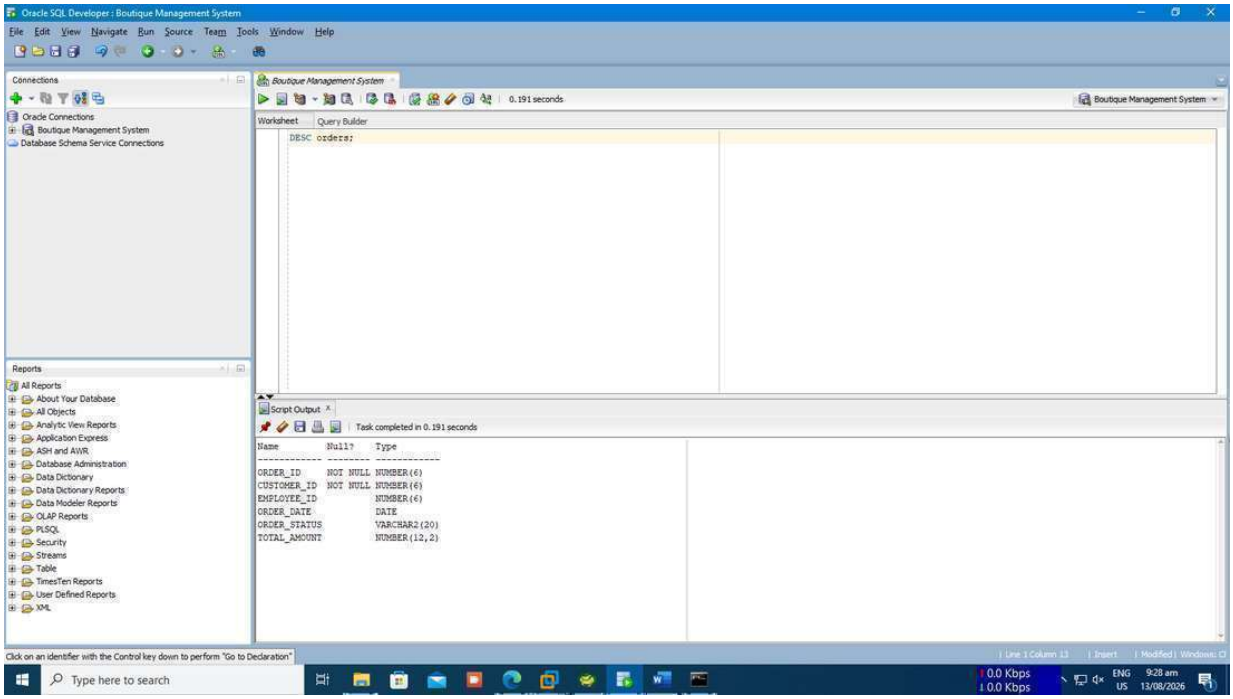
EMPLOYEES → ORDERS

Verification

```
DESC orders;
```

Figure 6.13: ORDERS Table Structure





Step 9: Create ORDER_DETAILS Table

```
CREATE TABLE order_details (
   order_detail_id NUMBER(8),
   order_id NUMBER(6) NOT NULL,
   product_id NUMBER(6) NOT NULL,
   quantity NUMBER(8) NOT NULL,
   unit_price NUMBER(10,2) NOT NULL,
   subtotal NUMBER(12,2) NOT NULL,
   CONSTRAINT pk_order_details PRIMARY KEY (order_detail_id),
   CONSTRAINT fk_detail_order
      FOREIGN KEY (order_id)
      REFERENCES orders(order_id),
   CONSTRAINT fk_detail_product
      FOREIGN KEY (product_id)
      REFERENCES products(product_id),
   CONSTRAINT chk_detail_quantity
      CHECK (quantity > 0),
   CONSTRAINT chk_detail_price
      CHECK (unit_price > 0),
   CONSTRAINT chk_detail_subtotal
      CHECK (subtotal >= 0)
);
```

Explanation

ORDER_DETAILS connects orders with products.

For example, one order can contain:

Order #1

├── Dress

├── Shawl

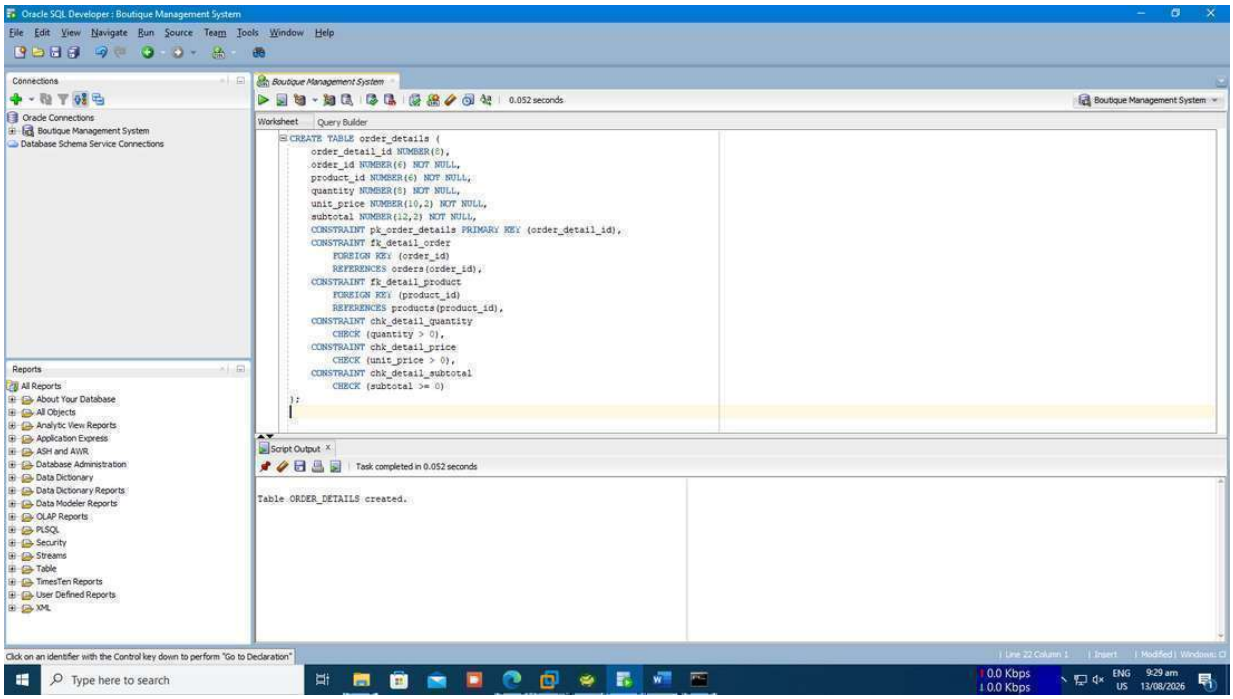
└── Suit

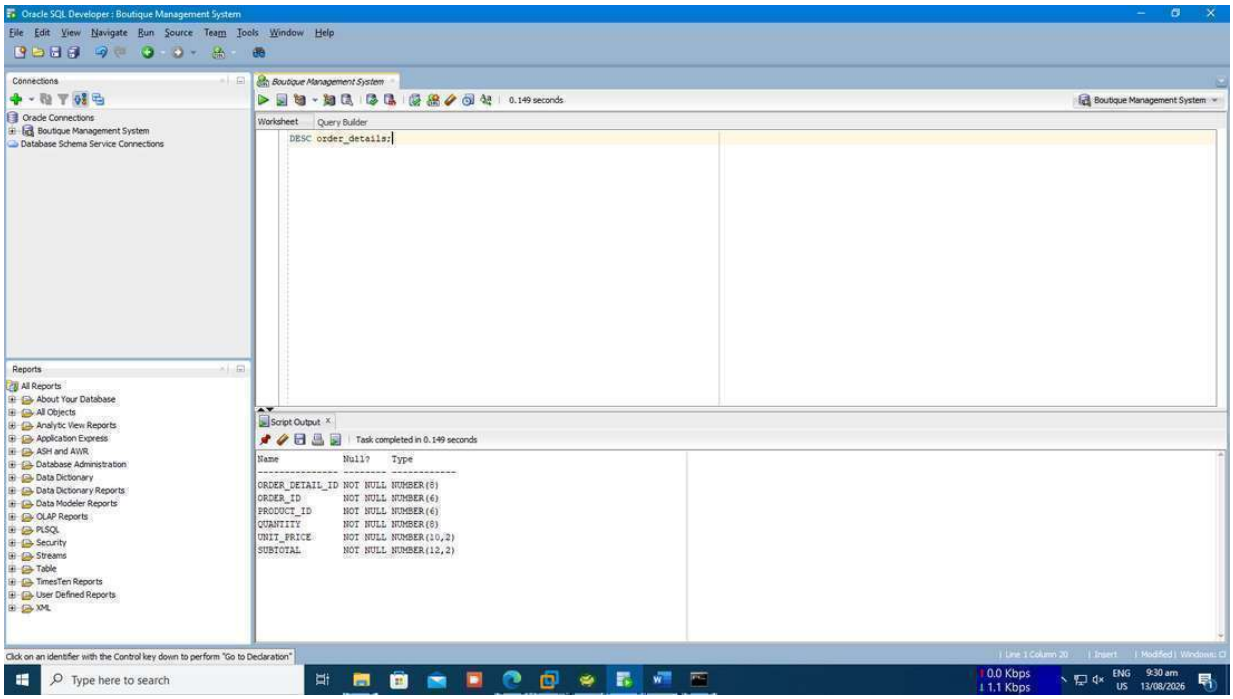
Therefore, this table is used to store individual products included in an order.

Verification

```
DESC order_details;
```

Figure 6.14: ORDER_DETAILS Table Structure





Step 10: Create PAYMENTS Table

```
CREATE TABLE payments (
    payment_id NUMBER(6),
    order_id NUMBER(6) NOT NULL,
    payment_date DATE DEFAULT SYSDATE,
    amount NUMBER(12,2) NOT NULL,
    payment_method VARCHAR2(30),
    payment_status VARCHAR2(20) DEFAULT 'PAID',
    CONSTRAINT pk_payments PRIMARY KEY (payment_id),
    CONSTRAINT fk_payment_order
        FOREIGN KEY (order_id)
        REFERENCES orders(order_id),
    CONSTRAINT chk_payment_amount
        CHECK (amount > 0)
);
```

Explanation

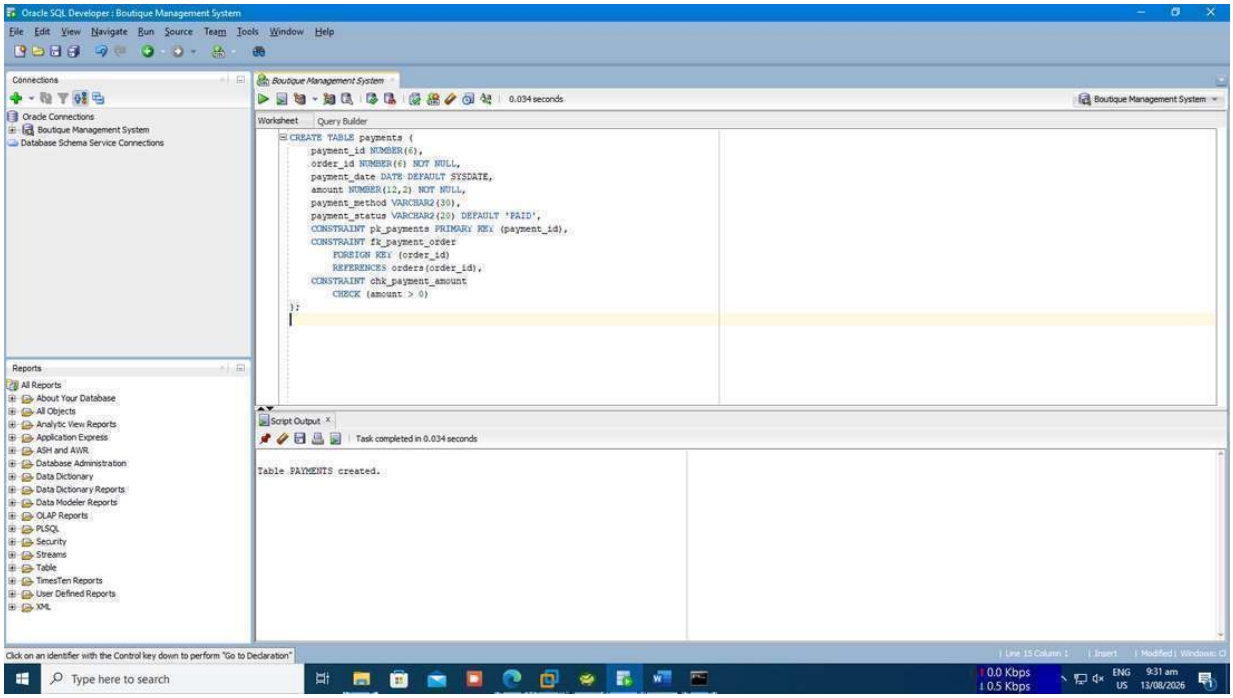
The PAYMENTS table stores payment information associated with customer orders.

The ORDER_ID is a foreign key referencing the ORDERS table.

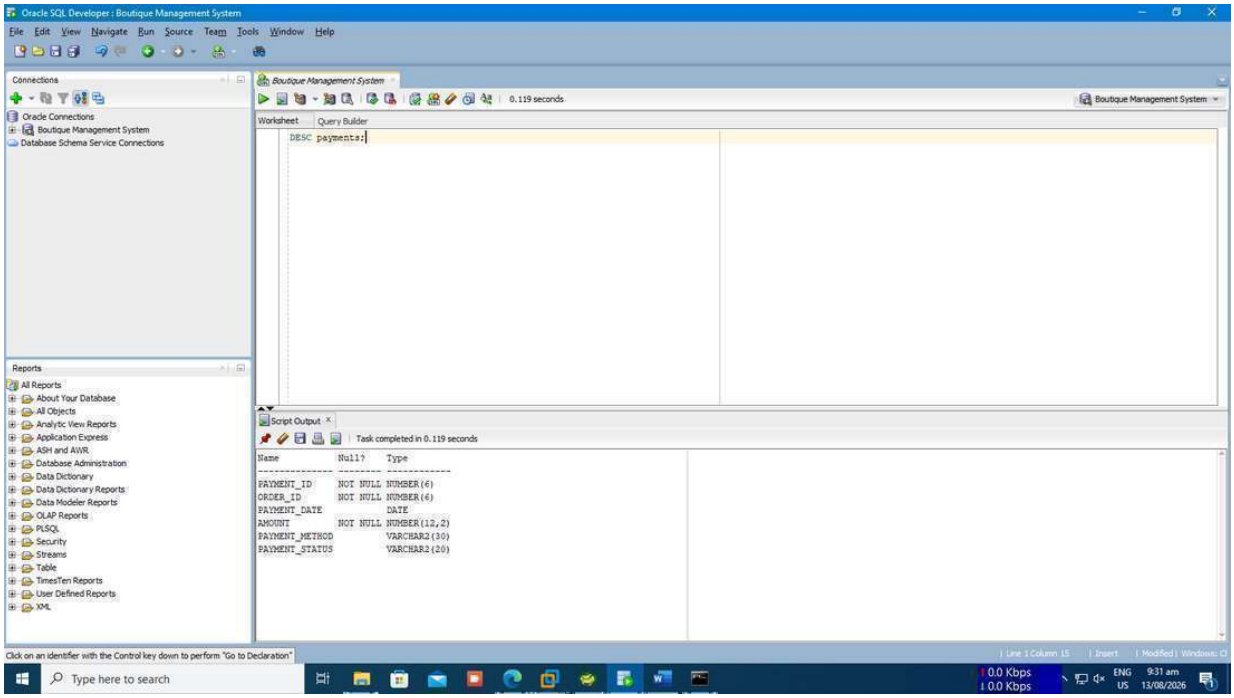
Verification

```
DESC payments;
```

Figure 6.15: PAYMENTS Table Structure



Step 11: Verify All Tables



Ab ye **important final verification** hai.

Run:

SELECT table_name

FROM user_tables

ORDER BY table_name;

Expected tables:

CATEGORIES

CUSTOMERS

EMPLOYEES

ORDER_DETAILS

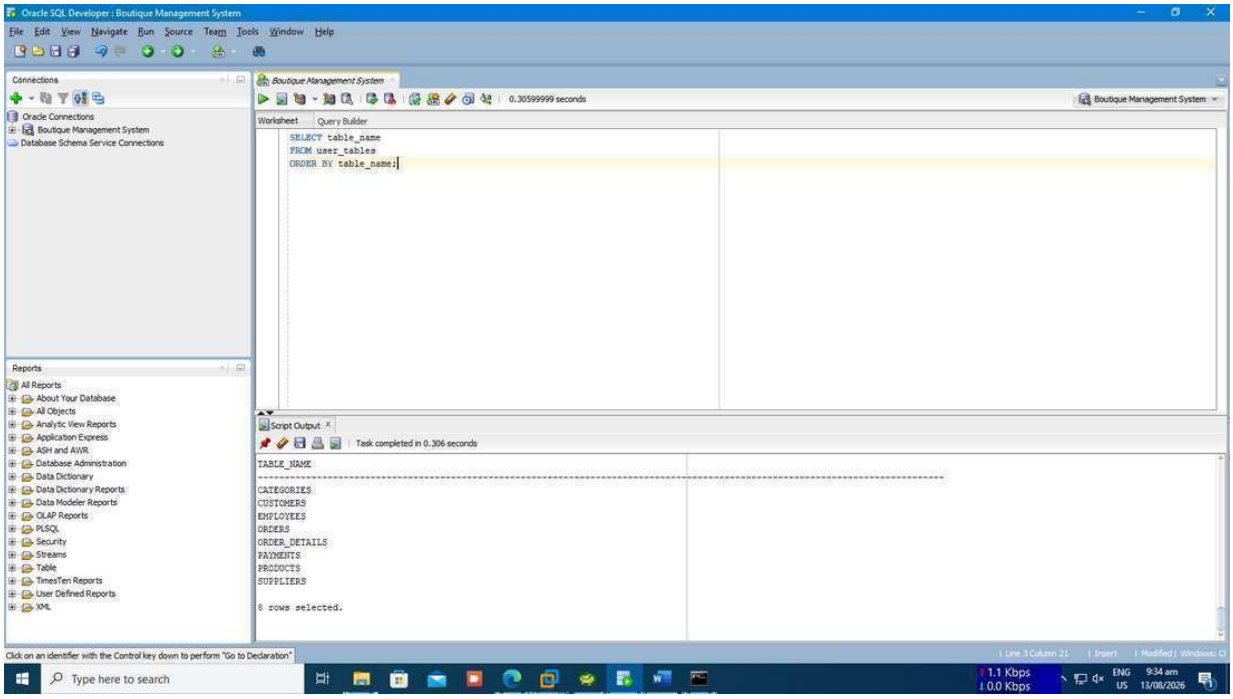
ORDERS

PAYMENTS

PRODUCTS

SUPPLIERS

Figure 6.16: All Boutique Management System Tables



6.10 Constraints Implementation

Objective

The objective of this step is to implement and verify database constraints in the Boutique Management System. Constraints help maintain **data integrity, accuracy, and consistency** within the database.

The project uses the following important constraints:

- Primary Key
- Foreign Key
- Unique
- NOT NULL
- CHECK

Step 1: Verify Primary Key Constraints

Primary keys uniquely identify each record in a table.

Run:

```
SELECT constraint_name, table_name, constraint_type
FROM user_constraints
WHERE constraint_type = 'P'
ORDER BY table_name;
```

Expected Output

You should see primary-key constraints similar to:

PK_CATEGORIES

PK_CUSTOMERS

PK_EMPLOYEES

PK_ORDER_DETAILS

PK_ORDERS

PK_PAYMENTS

PK_PRODUCTS

PK_SUPPLIERS

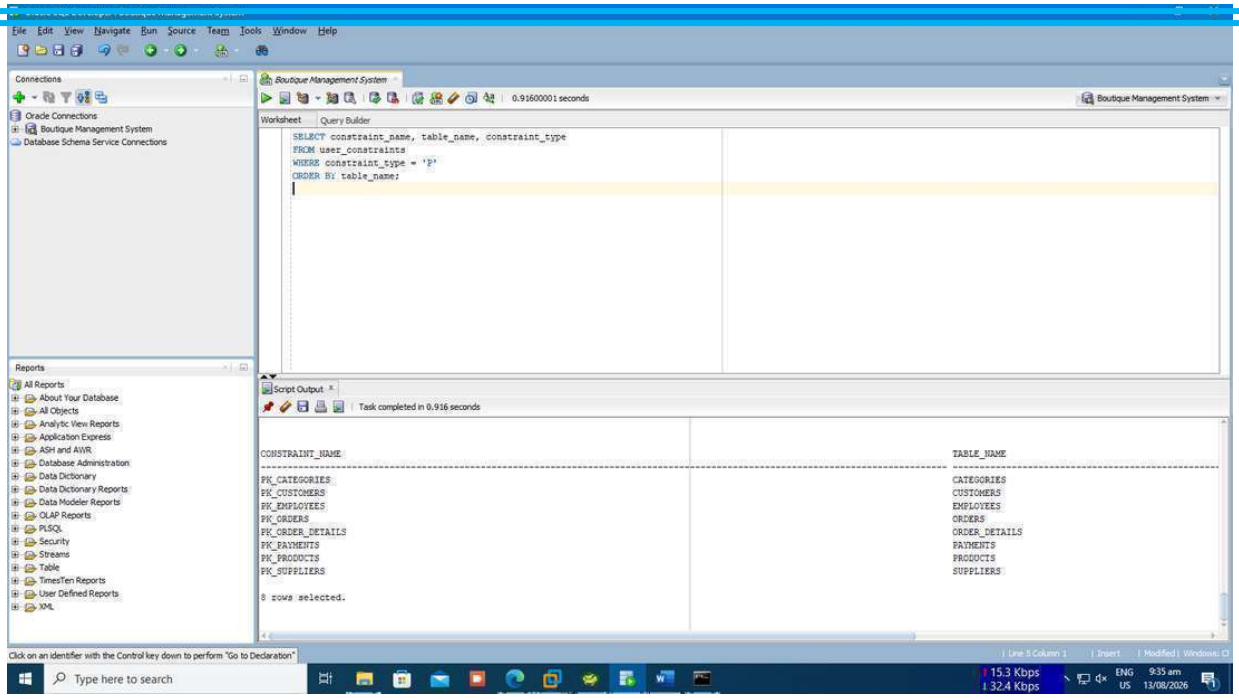
Explanation

The constraint type P represents a **Primary Key** in Oracle.

Each major table has its own primary key:

Table	**Primary Key**
CUSTOMERS	CUSTOMER_ID
EMPLOYEES	EMPLOYEE_ID
CATEGORIES	CATEGORY_ID
SUPPLIERS	SUPPLIER_ID
PRODUCTS	PRODUCT_ID
ORDERS	ORDER_ID
ORDER_DETAILS	ORDER_DETAIL_ID
PAYMENTS	PAYMENT_ID

Figure 6.17: Primary Key Constraints Verification



Step 2: Verify Foreign Key Constraints

Run:

```
SELECT constraint_name,
    table_name,
    r_constraint_name
FROM user_constraints
WHERE constraint_type = 'R'
ORDER BY table_name;
```

Explanation

The constraint type R represents a **Foreign Key**.

Foreign keys establish relationships between tables and prevent invalid references.

For example:

PRODUCTS.CATEGORY_ID

↓

CATEGORIES.CATEGORY_ID

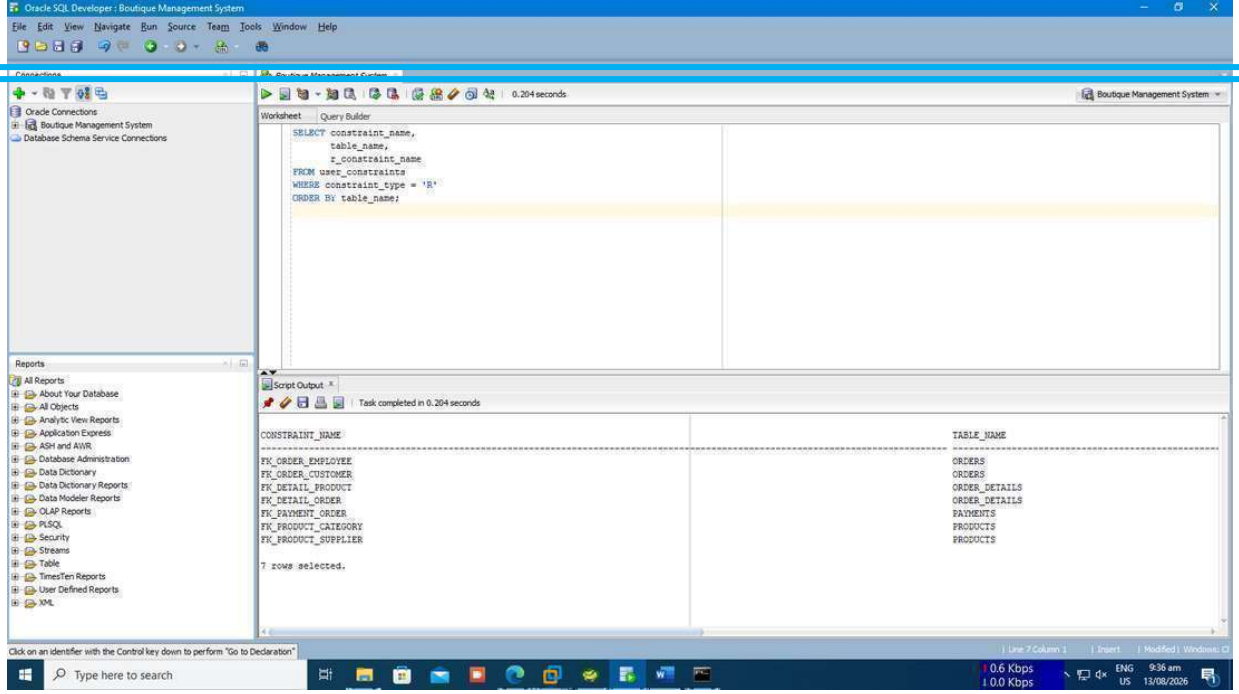
and:

ORDERS.CUSTOMER_ID

↓

CUSTOMERS.CUSTOMER_ID

Figure 6.18: Foreign Key Constraints Verification



Step 3: Verify UNIQUE Constraint

Our CATEGORIES table contains a unique constraint on CATEGORY_NAME.

Run:

```
SELECT constraint_name,
    table_name,
    constraint_type
FROM user_constraints
WHERE table_name = 'CATEGORIES';
```

Explanation

The UNIQUE constraint prevents duplicate category names from being stored in the table.

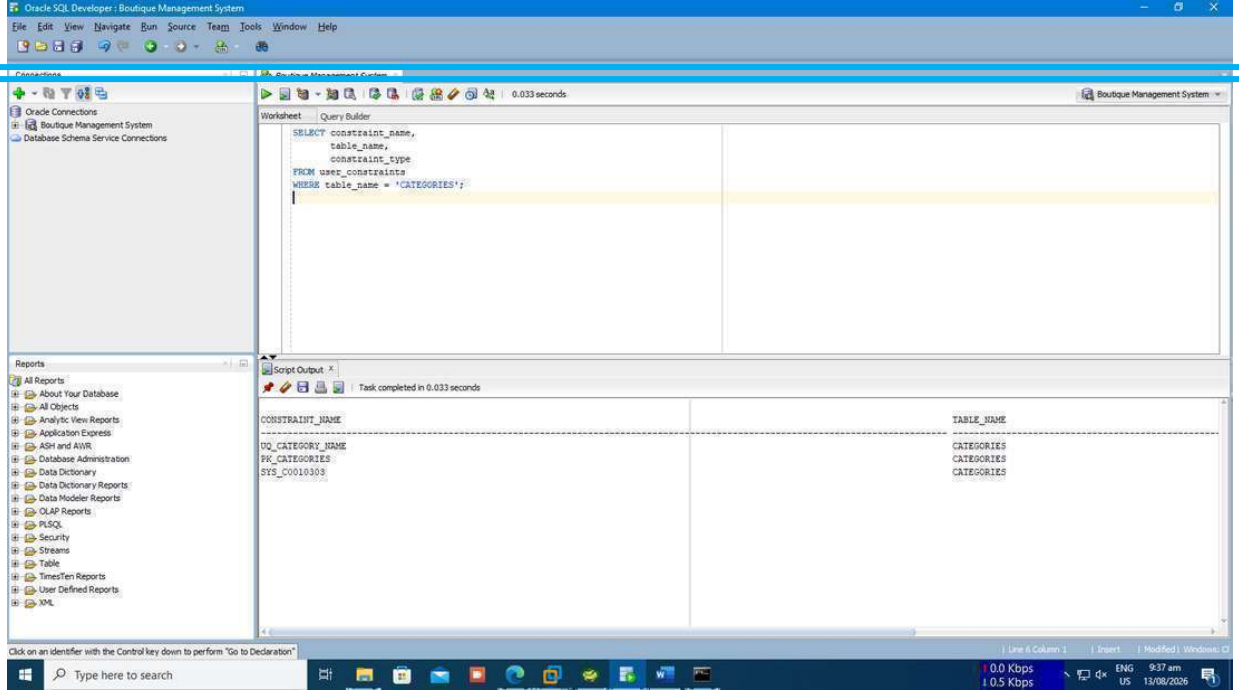
For example, the following two records cannot have the same category name:

1 Dresses

2 Dresses

Oracle will reject the second duplicate value.

Figure 6.19: UNIQUE Constraint Verification



Step 4: Verify NOT NULL Constraints

Run:

```
SELECT table_name,
    column_name,
    nullable
FROM user_tab_columns
WHERE table_name IN
('CUSTOMERS','EMPLOYEES','CATEGORIES','SUPPLIERS',
 'PRODUCTS','ORDERS','ORDER_DETAILS','PAYMENTS')
ORDER BY table_name, column_id;
```

Explanation

The NULLABLE column shows whether a column can contain NULL values.

For example, important fields such as:

CUSTOMER_NAME

EMPLOYEE_NAME

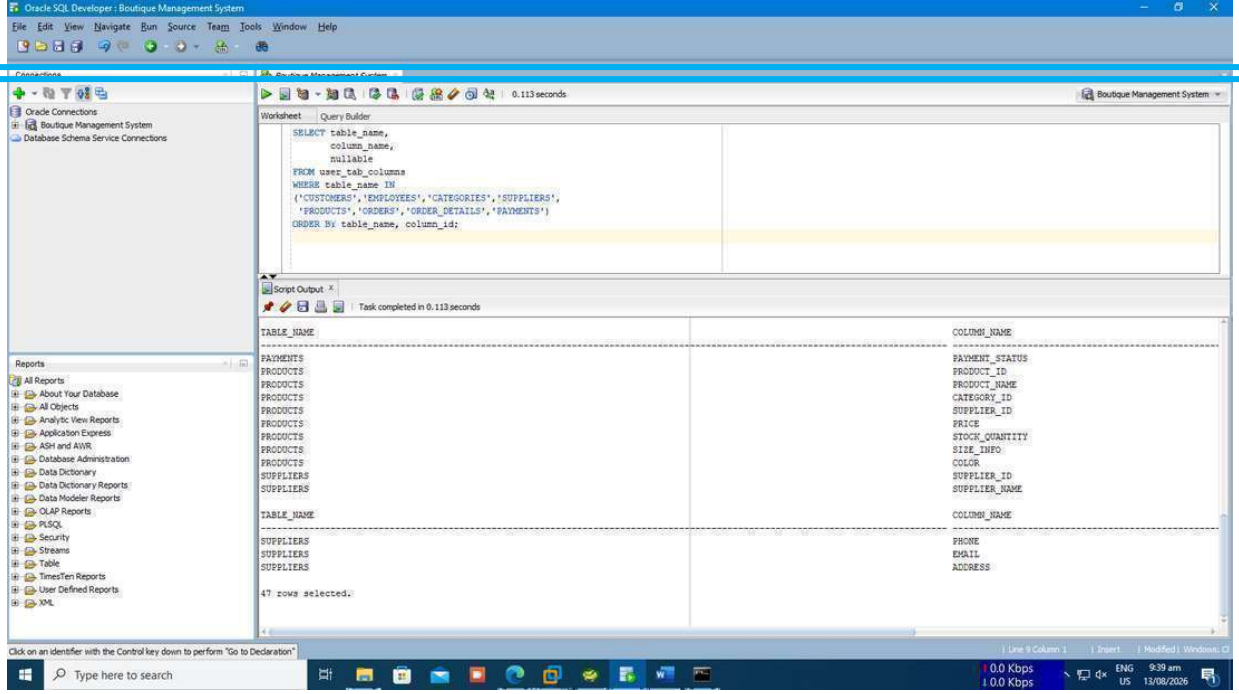
CATEGORY_NAME

PRODUCT_NAME

were defined with NOT NULL.

This ensures that required information cannot be left empty.

Figure 6.20: NOT NULL Constraint Verification



Step 5: Verify CHECK Constraints

Run:

```
SELECT constraint_name,
    table_name,
    search_condition
FROM user_constraints
WHERE constraint_type = 'C'
ORDER BY table_name;
```

Explanation

CHECK constraints restrict the values that can be entered into specific columns.

For example, in the PRODUCTS table:

CHECK (price > 0)

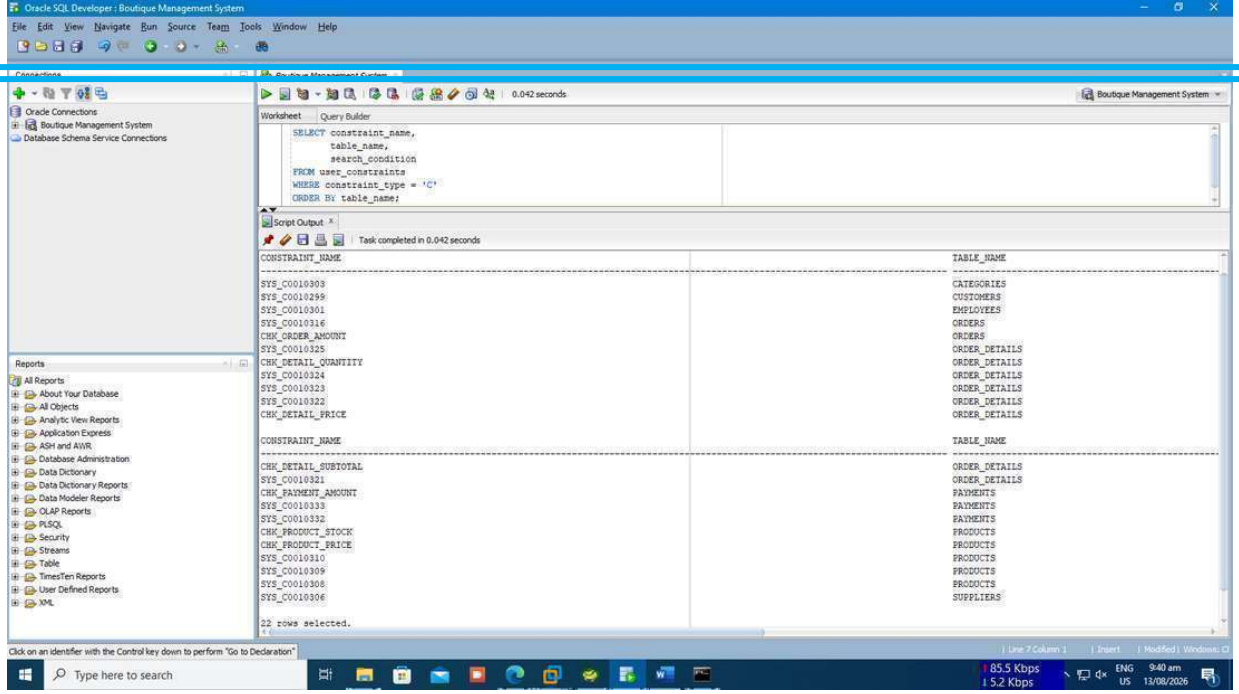
ensures that the product price cannot be zero or negative.

Similarly:

CHECK (stock_quantity >= 0)

prevents negative stock quantities.

Figure 6.21: CHECK Constraints Verification



Step 6: Display All Constraints Together

For the final verification of this section, run:

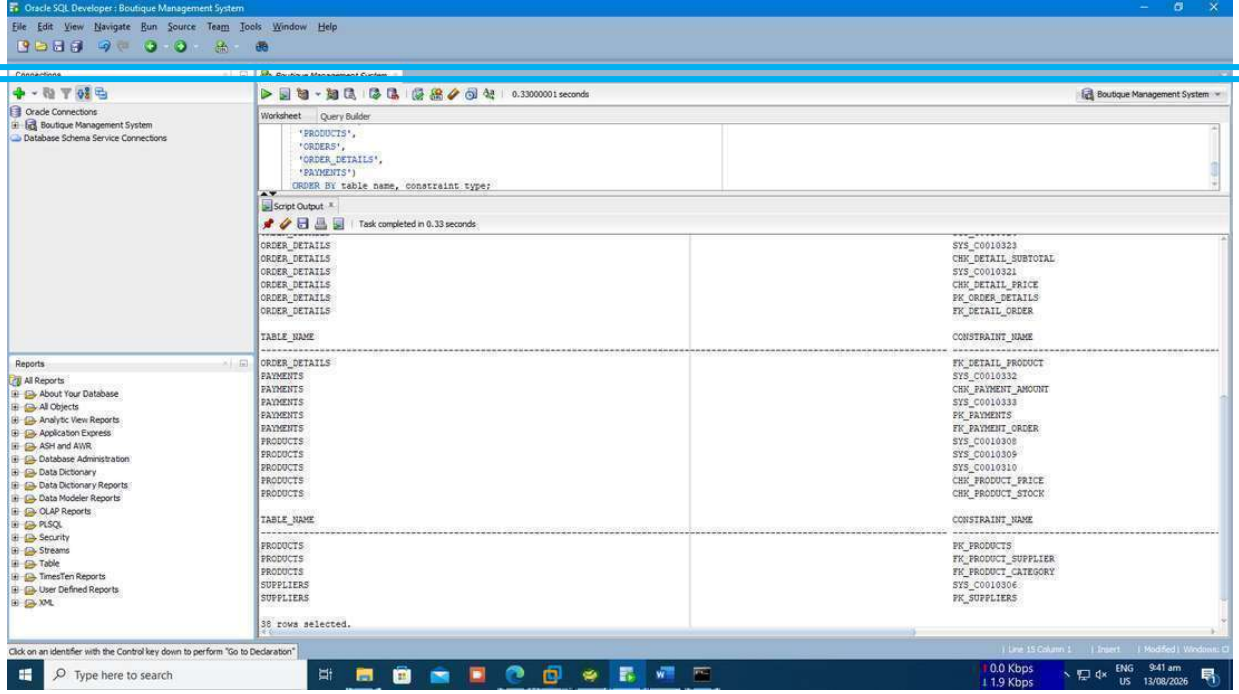
```
SELECT table_name,
    constraint_name,
    constraint_type
FROM user_constraints
WHERE table_name IN
('CUSTOMERS',
 'EMPLOYEES',
 'CATEGORIES',
 'SUPPLIERS',
 'PRODUCTS',
 'ORDERS',
 'ORDER_DETAILS',
 'PAYMENTS')
ORDER BY table_name, constraint_type;
```

Explanation

This query displays all constraints defined on the Boutique Management System tables.

It provides a centralized verification of the database integrity rules.

Figure 6.22: Complete Constraint Verification



Constraint Summary

Constraint	Purpose
PRIMARY KEY	Uniquely identifies each record
FOREIGN KEY	Establishes relationships between tables
UNIQUE	Prevents duplicate values
NOT NULL	Ensures required data is entered
CHECK	Restricts invalid values

Section Outcome

After completing this section, the Boutique Management System contains properly defined and verified constraints that help maintain **referential integrity and data consistency**.

Objective

The objective of this step is to create Oracle sequences for automatically generating unique identification numbers for the major tables of the Boutique Management System.

Oracle sequences help avoid manually entering IDs and reduce the possibility of duplicate identification numbers.

Step 1: Create CUSTOMER Sequence

Run:

```
CREATE SEQUENCE seq_customers
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

Explanation

This sequence will generate unique values for `CUSTOMER_ID`.

For example:

```
1
2
3
4
5
...
```

Step 2: Create EMPLOYEE Sequence

```
CREATE SEQUENCE seq_employees
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

This sequence will generate unique employee IDs.

Step 3: Create CATEGORY Sequence

```
CREATE SEQUENCE seq_categories
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

This sequence will generate unique category IDs.

Step 4: Create SUPPLIER Sequence

```
CREATE SEQUENCE seq_suppliers
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

This sequence will generate unique supplier IDs.

Step 5: Create PRODUCT Sequence

```
CREATE SEQUENCE seq_products
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

This sequence will generate unique product IDs.

Step 6: Create ORDER Sequence

```
CREATE SEQUENCE seq_orders
START WITH 1
INCREMENT BY 1
NOCACHE
```

```
NOCYCLE;
```

This sequence will generate unique order IDs.

Step 7: Create ORDER DETAILS Sequence

```
CREATE SEQUENCE seq_order_details
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

This sequence will generate unique order-detail IDs.

Step 8: Create PAYMENT Sequence

```
CREATE SEQUENCE seq_payments
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

This sequence will generate unique payment IDs.

Step 9: Verify All Sequences

After creating all sequences, run:

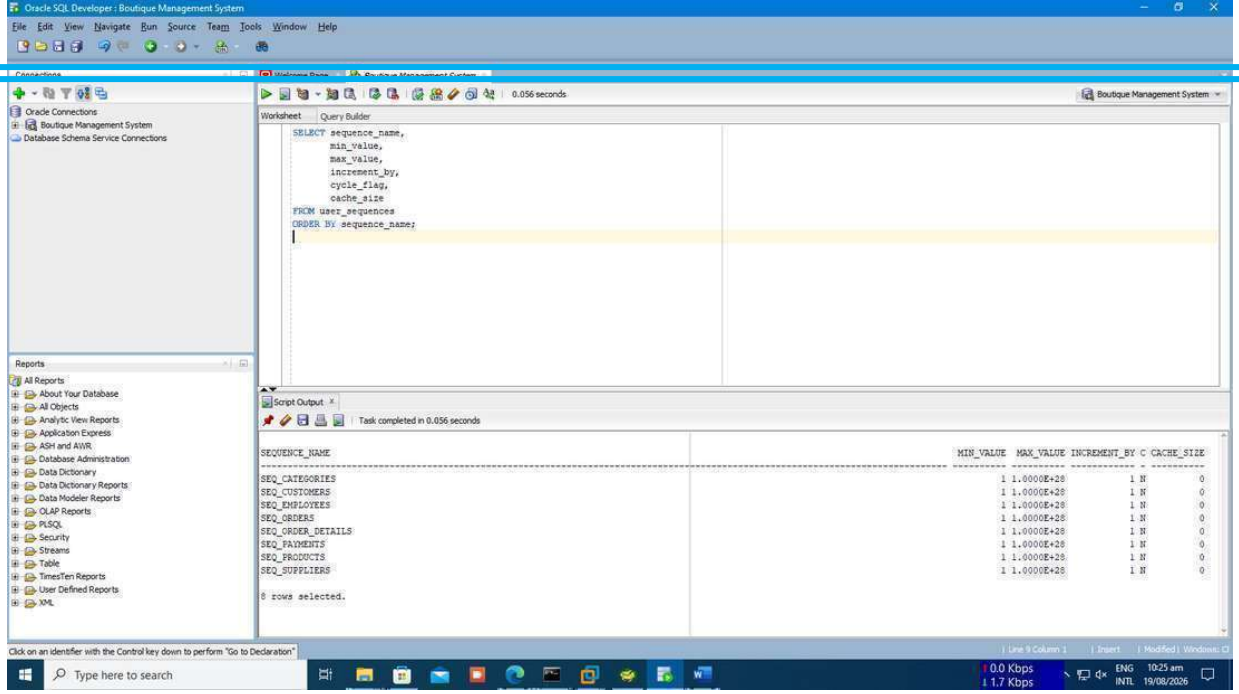
```
SELECT sequence_name,
       min_value,
       max_value,
       increment_by,
       cycle_flag,
       cache_size
FROM user_sequences
ORDER BY sequence_name;
```

Expected Output

You should see:

```
SEQ_CATEGORIES
SEQ_CUSTOMERS
SEQ_EMPLOYEES
SEQ_ORDER_DETAILS
SEQ_ORDERS
SEQ_PAYMENTS
SEQ_PRODUCTS
SEQ_SUPPLIERS
```

Figure 6.23: Boutique Management System Sequences



Step 10: Test a Sequence

For example, test the customer sequence:

```
SELECT seq_customers.NEXTVAL
FROM dual;
```

Expected:

```
1
```

Run it again:

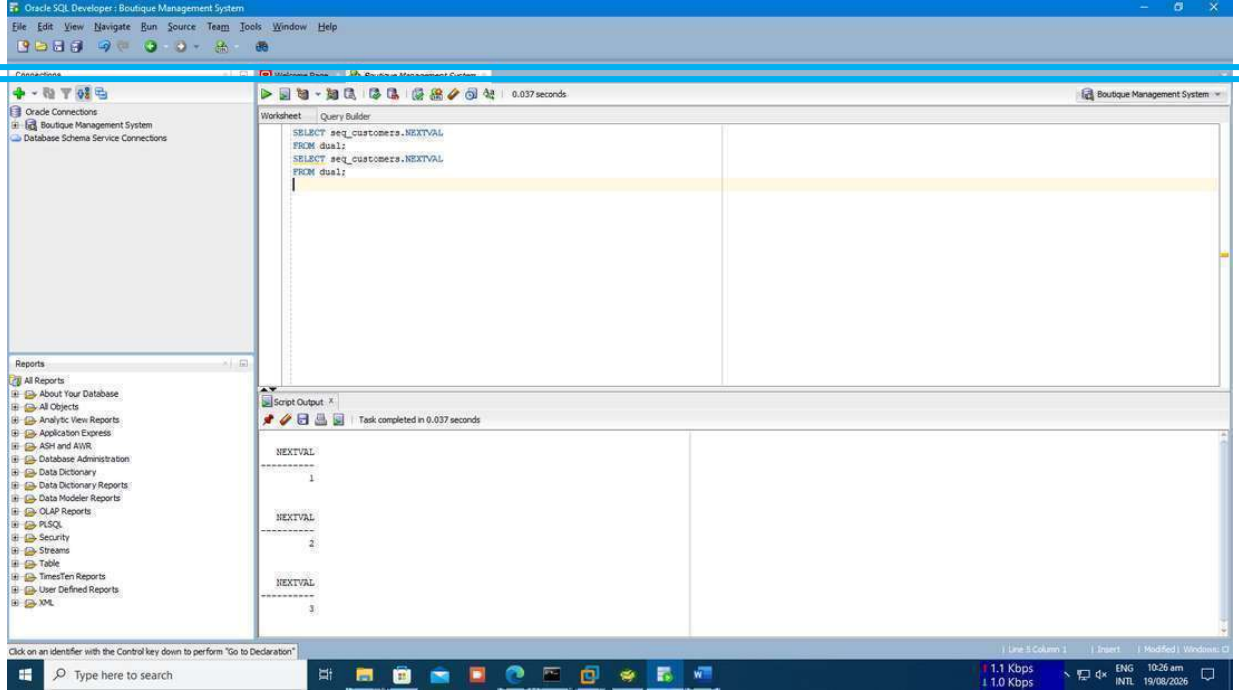
```
SELECT seq_customers.NEXTVAL
FROM dual;
```

Expected:

```
2
```

This confirms that the sequence automatically generates the next unique value.

Figure 6.24: Sequence NEXTVAL Verification



Explanation

`NEXTVAL` generates the next available value from an Oracle sequence.

Later, when we insert customer records, we can use:

```
seq_customers.NEXTVAL
```

instead of manually entering the customer ID.Section Outcome

6.12 Sample Data Insertion

Objective

The objective of this step is to insert sample business data into the tables of the Boutique Management System. The sample data will be used later for testing, order processing, payment management, and SQL queries.

We will use the previously created **Oracle sequences** to generate unique IDs.

Step 1: Insert Customer Records

Run:

```
INSERT INTO customers
(customer_id, customer_name, phone, email, address)
VALUES
(seq_customers.NEXTVAL, 'Ayesha Khan', '03001234567',
 'ayesha@gmail.com', 'Lahore');

INSERT INTO customers
(customer_id, customer_name, phone, email, address)
VALUES
(seq_customers.NEXTVAL, 'Sara Ahmed', '03111234567',
 'sara@gmail.com', 'Islamabad');

INSERT INTO customers
(customer_id, customer_name, phone, email, address)
VALUES
(seq_customers.NEXTVAL, 'Hina Malik', '03221234567',
 'hina@gmail.com', 'Rawalpindi');

INSERT INTO customers
(customer_id, customer_name, phone, email, address)
VALUES
(seq_customers.NEXTVAL, 'Maham Ali', '03331234567',
 'maham@gmail.com', 'Karachi');

INSERT INTO customers
(customer_id, customer_name, phone, email, address)
VALUES
(seq_customers.NEXTVAL, 'Zainab Shah', '03441234567',
 'zainab@gmail.com', 'Peshawar');
```

Explanation

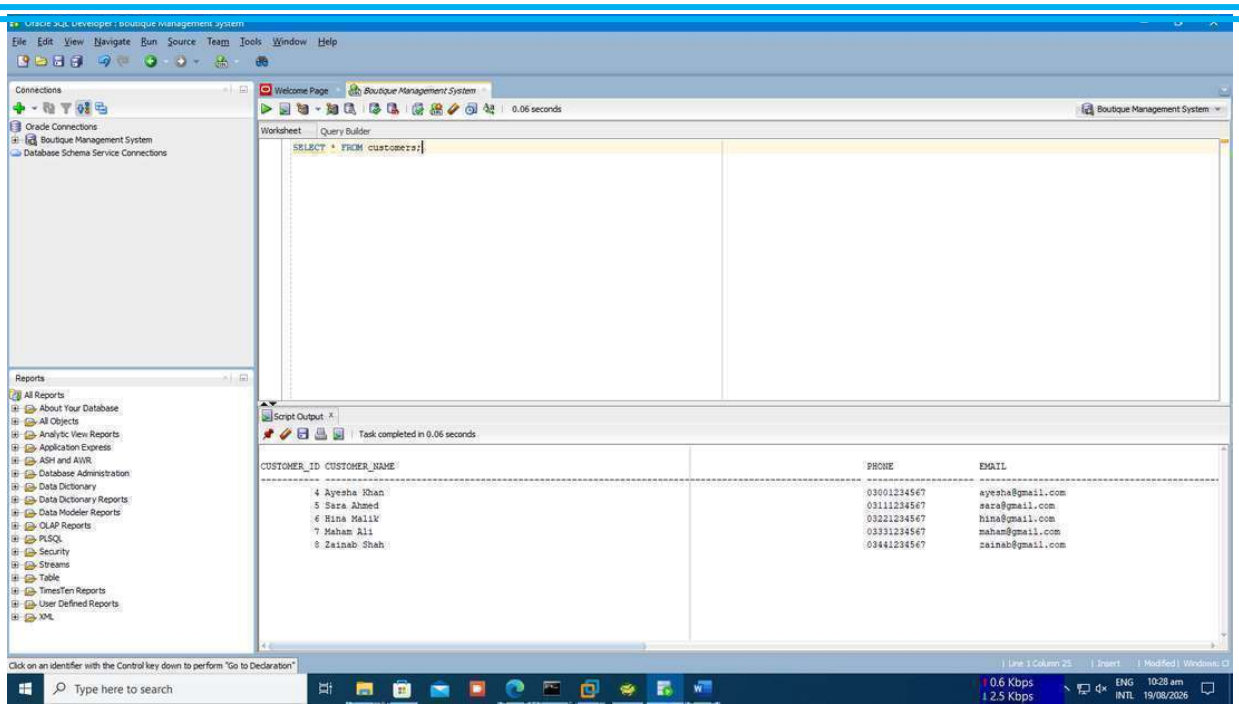
Five customer records are inserted into the `CUSTOMERS` table.

The `SEQ_CUSTOMERS.NEXTVAL` sequence automatically generates customer IDs.

Verification

```
SELECT * FROM customers;
```

Figure 6.25: Customer Sample Data



Step 2: Insert Employee Records

```
INSERT INTO employees
(employee_id, employee_name, phone, email, job_title, salary)
VALUES
(seq_employees.NEXTVAL, 'Fatima Noor', '03001112233',
 'fatima@boutique.com', 'Manager', 75000);

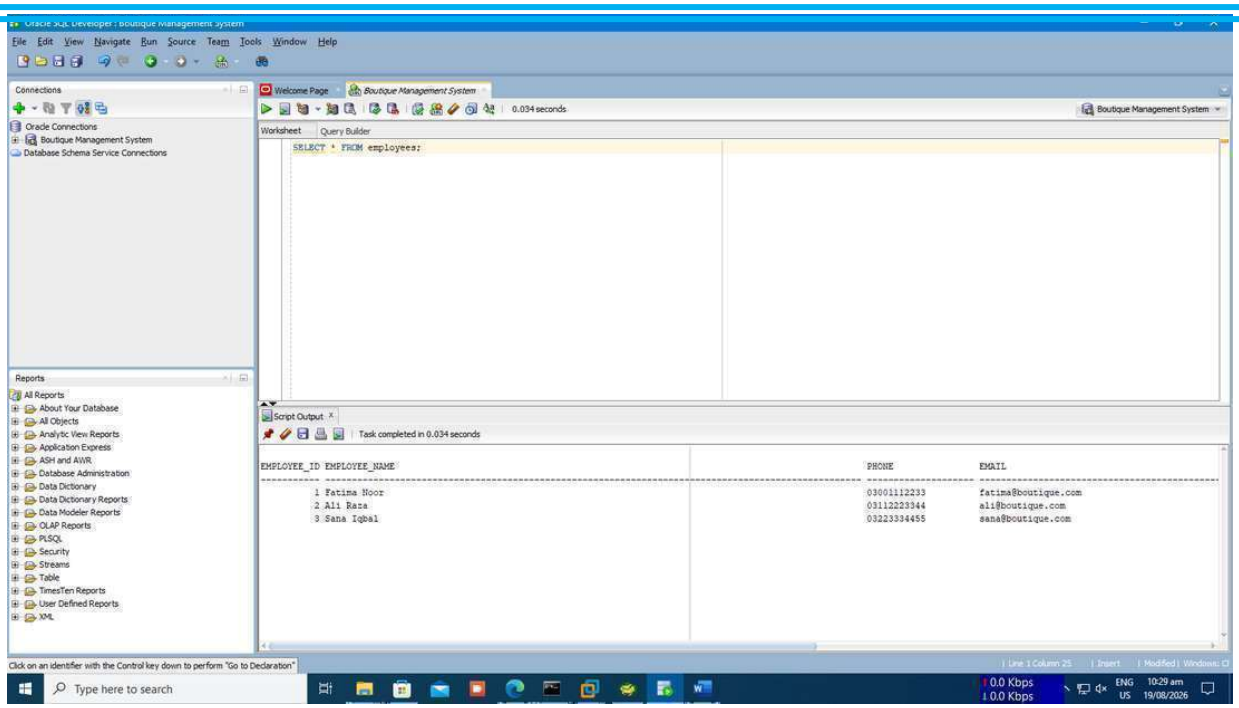
INSERT INTO employees
(employee_id, employee_name, phone, email, job_title, salary)
VALUES
(seq_employees.NEXTVAL, 'Ali Raza', '03112223344',
 'ali@boutique.com', 'Sales Officer', 45000);

INSERT INTO employees
(employee_id, employee_name, phone, email, job_title, salary)
VALUES
(seq_employees.NEXTVAL, 'Sana Iqbal', '03223334455',
 'sana@boutique.com', 'Sales Officer', 45000);
```

Verification

```
SELECT * FROM employees;
```

Figure 6.26: Employee Sample Data



Step 3: Insert Category Records

```
INSERT INTO categories
(category_id, category_name, description)
VALUES
(seq_categories.NEXTVAL, 'Dresses', 'Women formal and casual dresses');

INSERT INTO categories
(category_id, category_name, description)
VALUES
(seq_categories.NEXTVAL, 'Suits', 'Designer and formal suits');

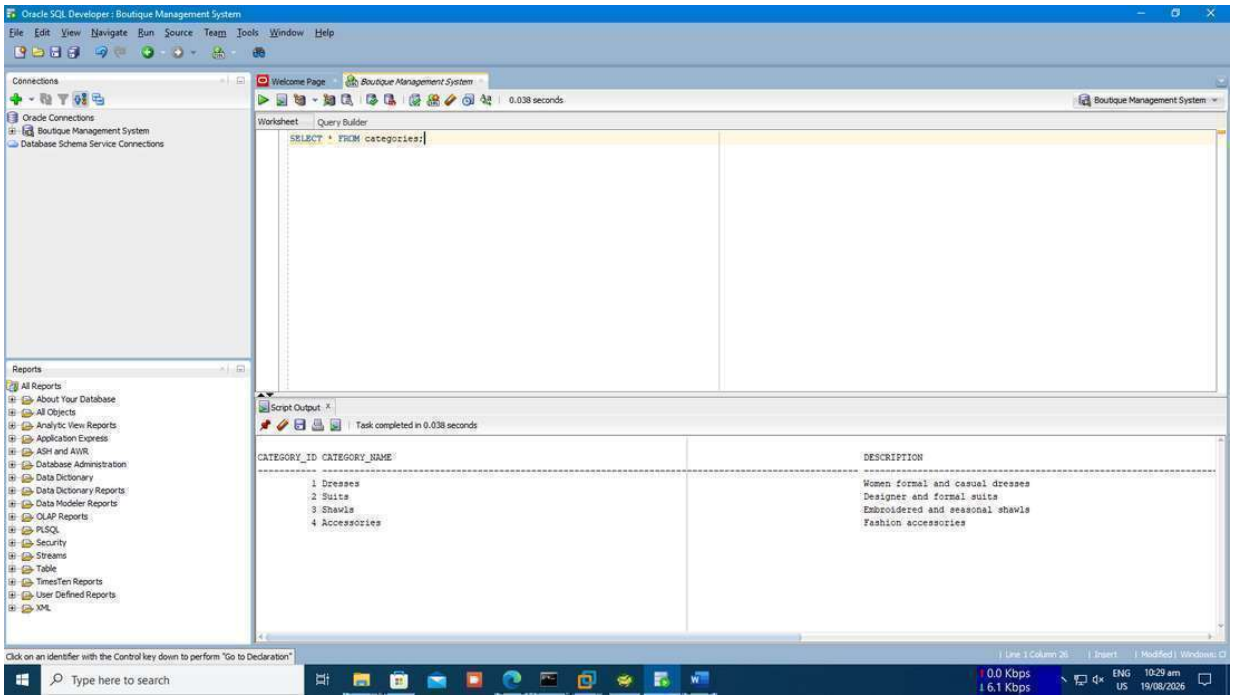
INSERT INTO categories
(category_id, category_name, description)
VALUES
(seq_categories.NEXTVAL, 'Shawls', 'Embroidered and seasonal shawls');

INSERT INTO categories
(category_id, category_name, description)
VALUES
(seq_categories.NEXTVAL, 'Accessories', 'Fashion accessories');
```

Verification

```
SELECT * FROM categories;
```

Figure 6.27: Category Sample Data



Step 4: Insert Supplier Records

```
INSERT INTO suppliers
(supplier_id, supplier_name, phone, email, address)
VALUES
(seq_suppliers.NEXTVAL, 'Elegant Fabrics', '03005556677',
 'info@elegantfabrics.com', 'Lahore');

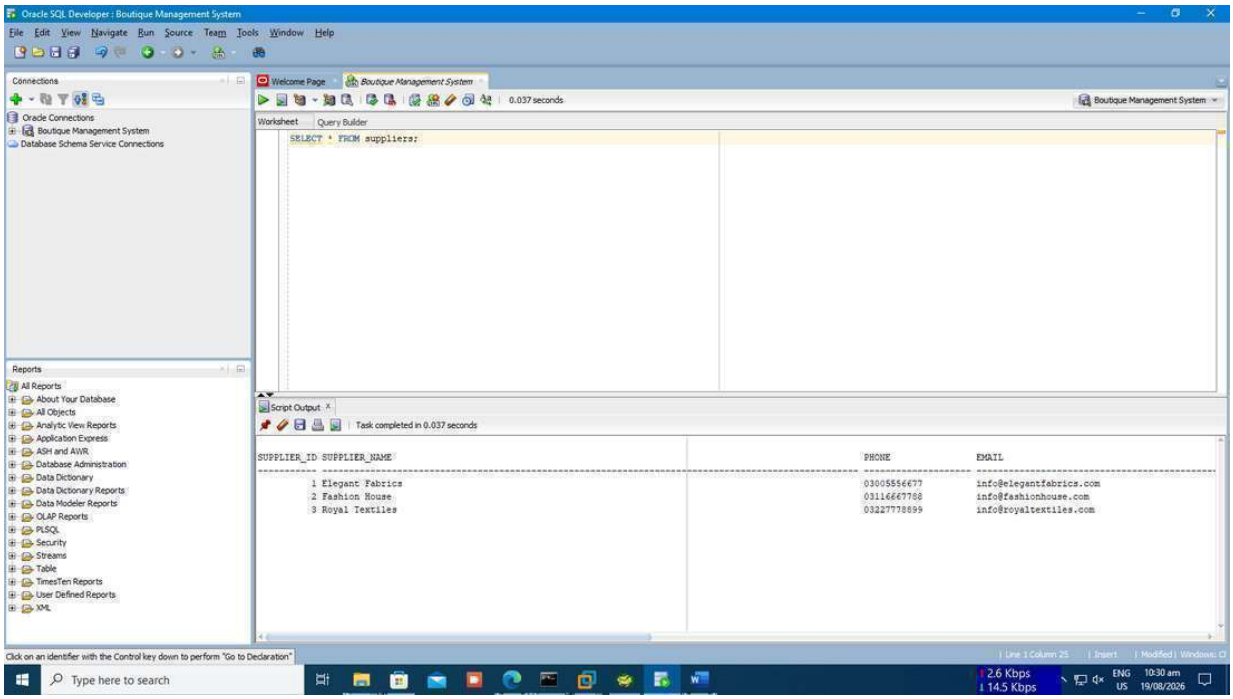
INSERT INTO suppliers
(supplier_id, supplier_name, phone, email, address)
VALUES
(seq_suppliers.NEXTVAL, 'Fashion House', '03116667788',
 'info@fashionhouse.com', 'Karachi');

INSERT INTO suppliers
(supplier_id, supplier_name, phone, email, address)
VALUES
(seq_suppliers.NEXTVAL, 'Royal Textiles', '03227778899',
 'info@royaltextiles.com', 'Faisalabad');
```

Verification

```
SELECT * FROM suppliers;
```

Figure 6.28: Supplier Sample Data



Step 5: Insert Product Records

Ab categories aur suppliers ke IDs use karte hue products insert karenge.

Pehle IDs verify kar lo:

```
SELECT category_id, category_name
FROM categories
ORDER BY category_id;
SELECT supplier_id, supplier_name
FROM suppliers
ORDER BY supplier_id;
```

Agar IDs **1, 2, 3, 4** aur suppliers ke IDs **1, 2, 3** hain, to ye records insert karo:

```
INSERT INTO products
(product_id, product_name, category_id, supplier_id,
 price, stock_quantity, size_info, color)
VALUES
(seq_products.NEXTVAL, 'Embroidered Lawn Dress',
 1, 1, 8500, 10, 'Medium', 'Pink');

INSERT INTO products
(product_id, product_name, category_id, supplier_id,
 price, stock_quantity, size_info, color)
VALUES
(seq_products.NEXTVAL, 'Formal Silk Suit',
 2, 2, 12500, 8, 'Large', 'Black');

INSERT INTO products
(product_id, product_name, category_id, supplier_id,
 price, stock_quantity, size_info, color)
VALUES
(seq_products.NEXTVAL, 'Embroidered Shawl',
 3, 3, 4500, 15, 'Standard', 'Maroon');

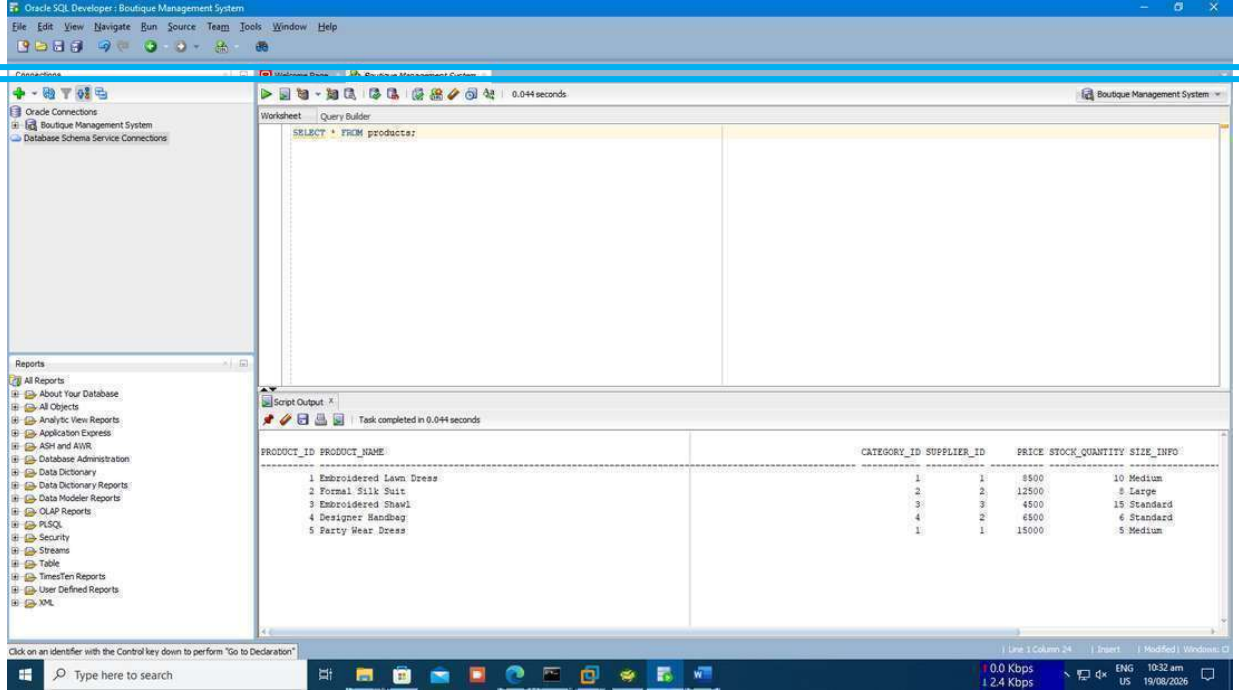
INSERT INTO products
(product_id, product_name, category_id, supplier_id,
 price, stock_quantity, size_info, color)
VALUES
(seq_products.NEXTVAL, 'Designer Handbag',
 4, 2, 6500, 6, 'Standard', 'Brown');

INSERT INTO products
(product_id, product_name, category_id, supplier_id,
 price, stock_quantity, size_info, color)
VALUES
(seq_products.NEXTVAL, 'Party Wear Dress',
 1, 1, 15000, 5, 'Medium', 'Blue');
```

Verification

```
SELECT * FROM products;
```

Figure 6.29: Product Sample Data



Step 6: Verify Relationships

Ab ek useful query run karte hain jo products ke saath category aur supplier ka naam bhi show karegi:

```
SELECT p.product_id,
       p.product_name,
       c.category_name,
       s.supplier_name,
       p.price,
       p.stock_quantity
FROM products p
JOIN categories c
     ON p.category_id = c.category_id
JOIN suppliers s
     ON p.supplier_id = s.supplier_id
ORDER BY p.product_id;
```

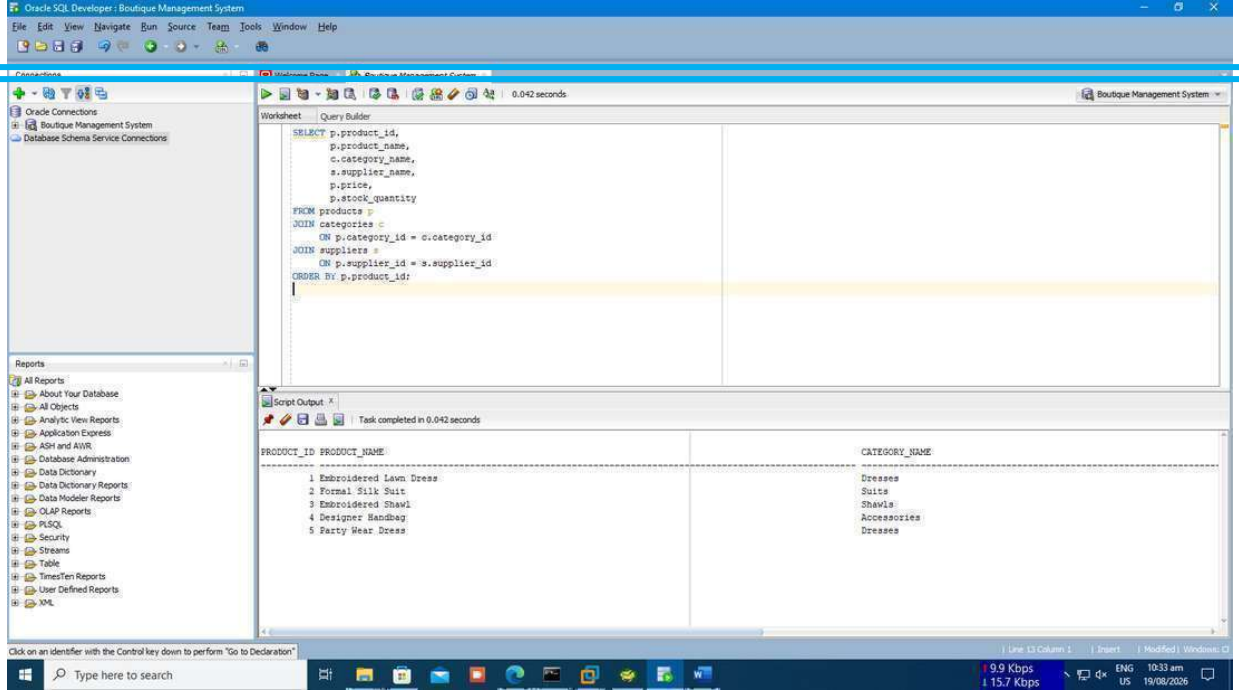
Explanation

This query demonstrates that the relationships between:

```
PRODUCTS → CATEGORIES
PRODUCTS → SUPPLIERS
```

are working correctly through foreign keys.

Figure 6.30: Product, Category and Supplier Relationship Verification



Step 7: Insert All Data Permanently

After verifying that all records are correct, run:

```
COMMIT;
```

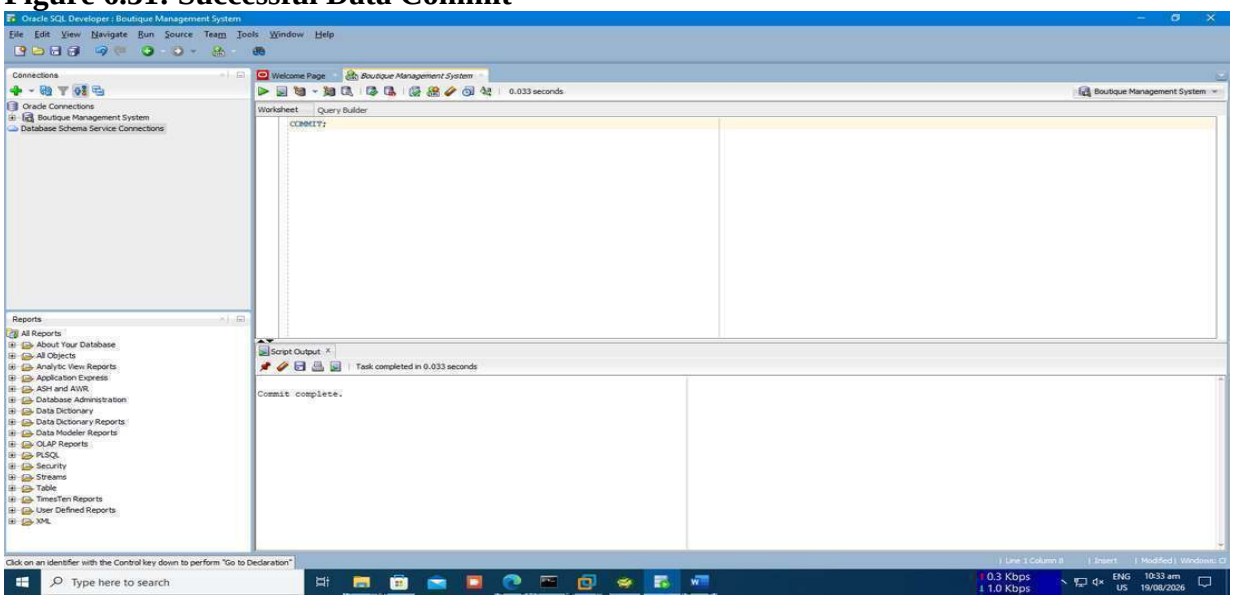
Explanation

The `COMMIT` command permanently saves the inserted records in the database.

Expected output:

```
Commit complete.
```

Figure 6.31: Successful Data Commit



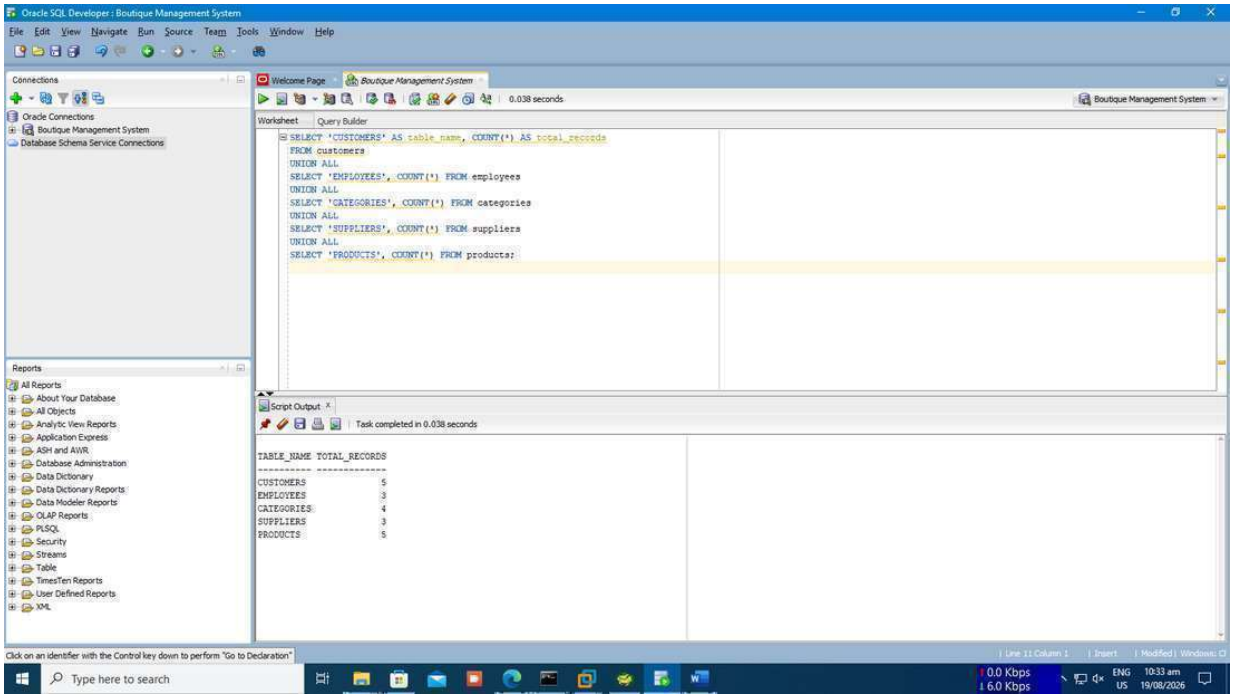
Step 8: Final Data Verification

Run:

```
SELECT 'CUSTOMERS' AS table_name, COUNT(*) AS total_records
FROM customers
UNION ALL
SELECT 'EMPLOYEES', COUNT(*) FROM employees
UNION ALL
SELECT 'CATEGORIES', COUNT(*) FROM categories
UNION ALL
SELECT 'SUPPLIERS', COUNT(*) FROM suppliers
UNION ALL
SELECT 'PRODUCTS', COUNT(*) FROM products;
```

This gives us a quick record count for the main master tables.

Figure 6.32: Sample Data Record Count Verification



6.14 SQL Queries

Objective

The objective of this section is to develop SQL queries for retrieving useful information from the Boutique Management System database.

SQL queries allow the database administrator and users to search, filter, summarize, and analyze stored information.

Step 1: Display All Customers

Command

```
SELECT *
FROM customers
ORDER BY customer_id;
```

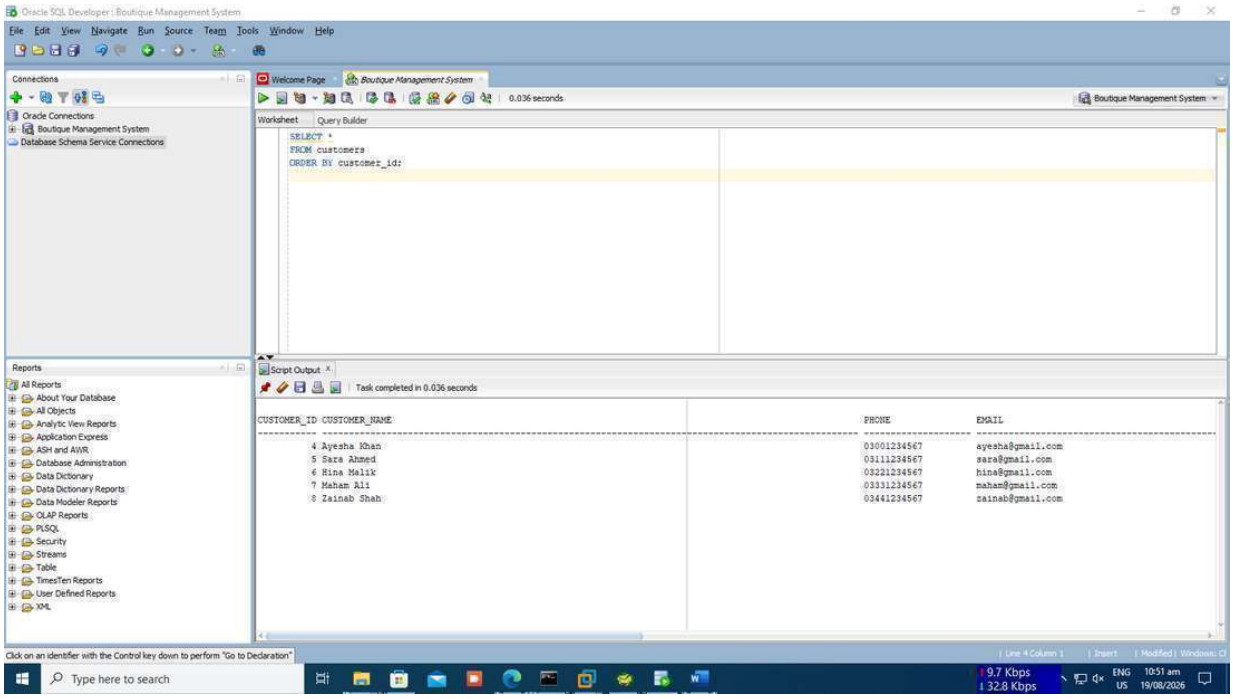
Explanation

This query displays all customer records stored in the `CUSTOMERS` table.

Verification

The query should display customer ID, name, phone, email, address, and registration date.

Figure 6.43: Displaying All Customer Records



Step 2: Display All Products

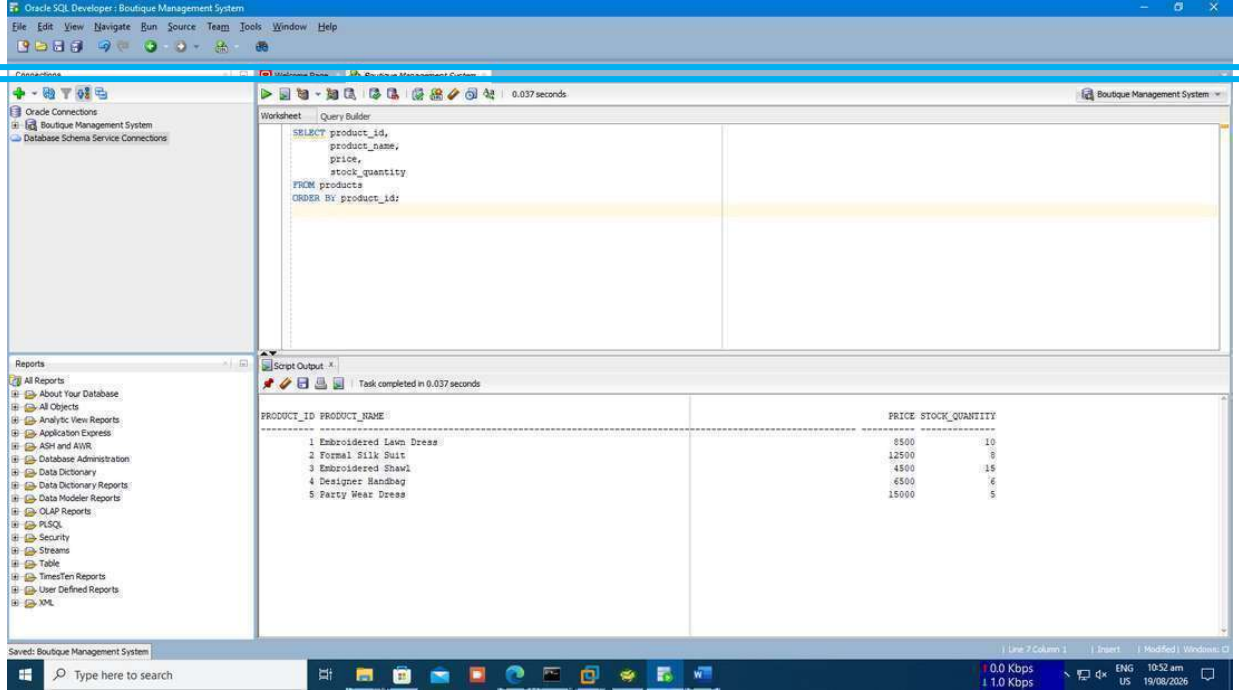
Command

```
SELECT product_id,
       product_name,
       price,
       stock_quantity
FROM products
ORDER BY product_id;
```

Explanation

This query displays the products available in the boutique along with their prices and stock quantities.

Figure 6.44: Product Information



Step 3: Display Products with Their Categories

Command

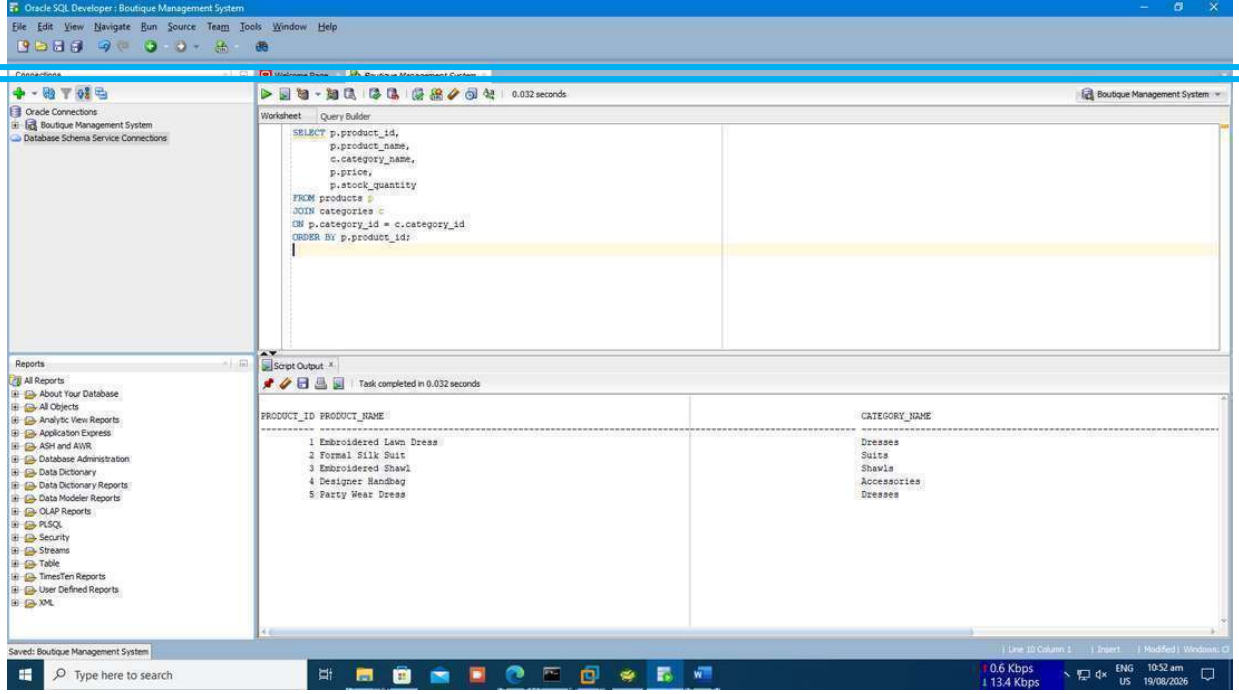
```
SELECT p.product_id,
       p.product_name,
       c.category_name,
       p.price,
       p.stock_quantity
FROM products p
JOIN categories c
ON p.category_id = c.category_id
ORDER BY p.product_id;
```

Explanation

This query uses an `INNER JOIN` to combine the `PRODUCTS` and `CATEGORIES` tables.

It displays each product along with its category.

Figure 6.45: Products with Categories



Step 4: Display Products with Supplier Information

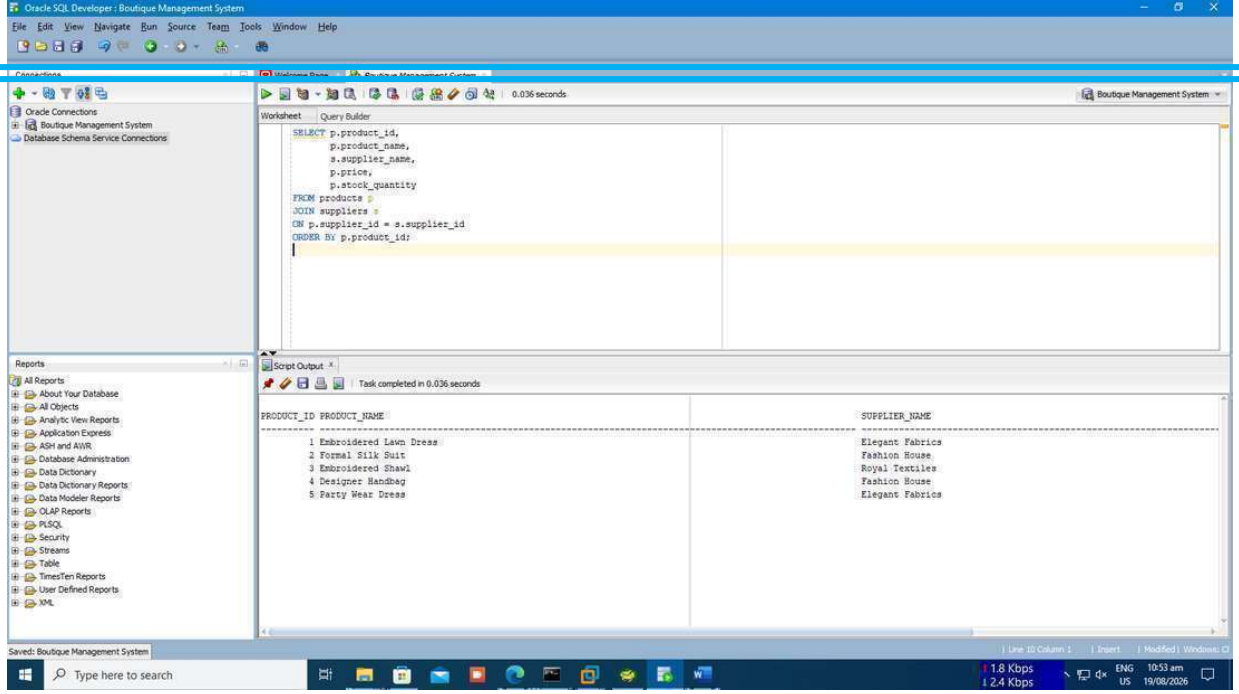
Command

```
SELECT p.product_id,
       p.product_name,
       s.supplier_name,
       p.price,
       p.stock_quantity
FROM products p
JOIN suppliers s
ON p.supplier_id = s.supplier_id
ORDER BY p.product_id;
```

Explanation

This query combines the `PRODUCTS` and `SUPPLIERS` tables to display the supplier of each product.

Figure 6.46: Products and Suppliers



Step 5: Display Products with Low Stock

Command

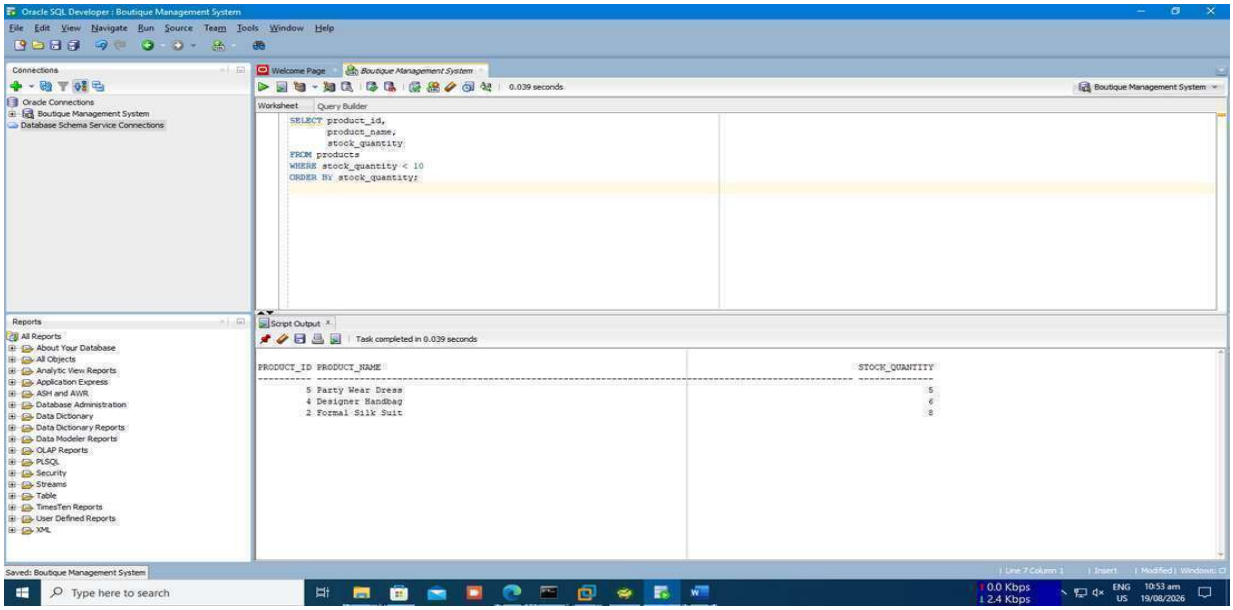
```
SELECT product_id,
       product_name,
       stock_quantity
FROM products
WHERE stock_quantity < 10
ORDER BY stock_quantity;
```

Explanation

This query identifies products whose stock quantity is less than 10.

This can help boutique management identify products that may need restocking.

Figure 6.47: Low Stock Products



Step 6: Display Customer Orders

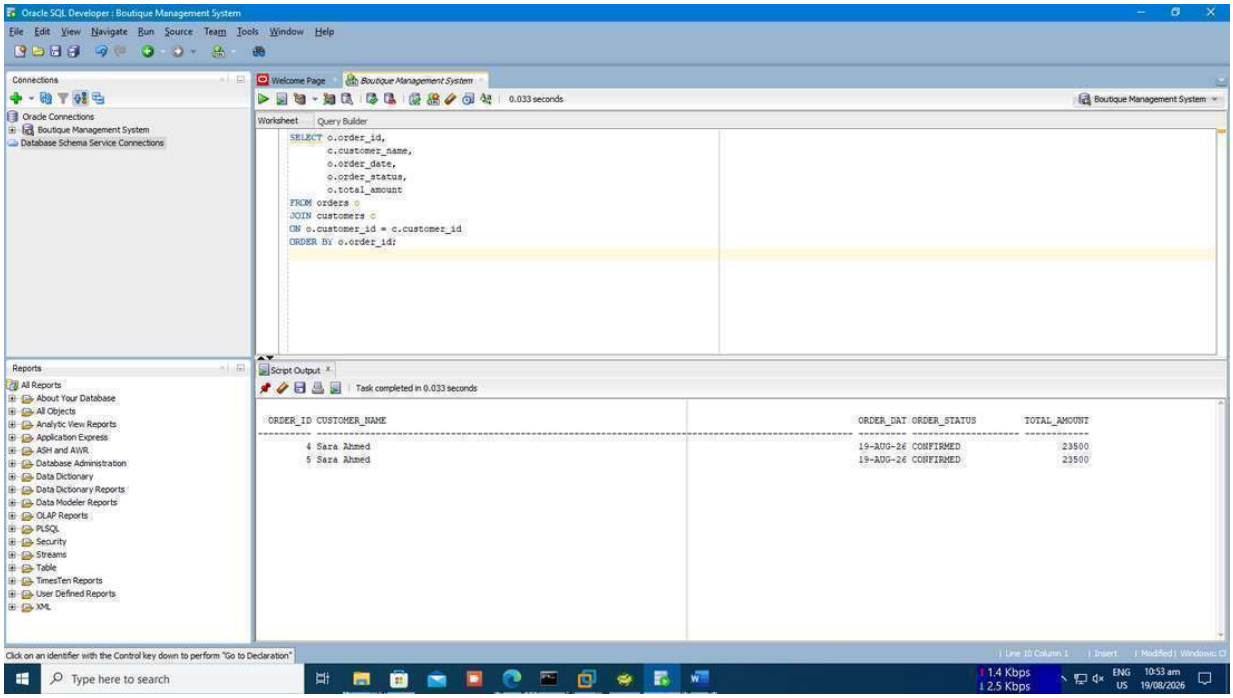
Command

```
SELECT o.order_id,
       c.customer_name,
       o.order_date,
       o.order_status,
       o.total_amount
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
ORDER BY o.order_id;
```

Explanation

This query displays order information together with the corresponding customer name.

Figure 6.48: Customer Orders



Step 7: Display Payment Information

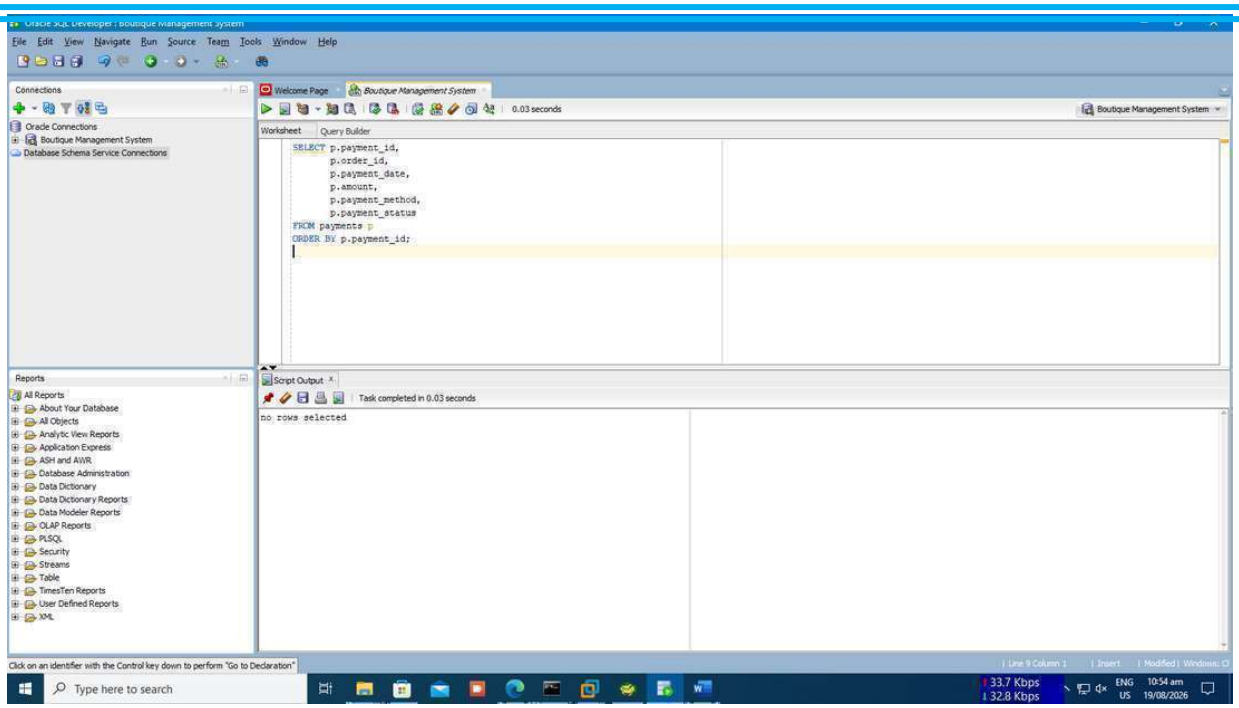
Command

```
SELECT p.payment_id,
       p.order_id,
       p.payment_date,
       p.amount,
       p.payment_method,
       p.payment_status
FROM payments p
ORDER BY p.payment_id;
```

Explanation

This query displays payment information stored in the `PAYMENTS` table.

Figure 6.49: Payment Information



Step 8: Calculate Total Sales

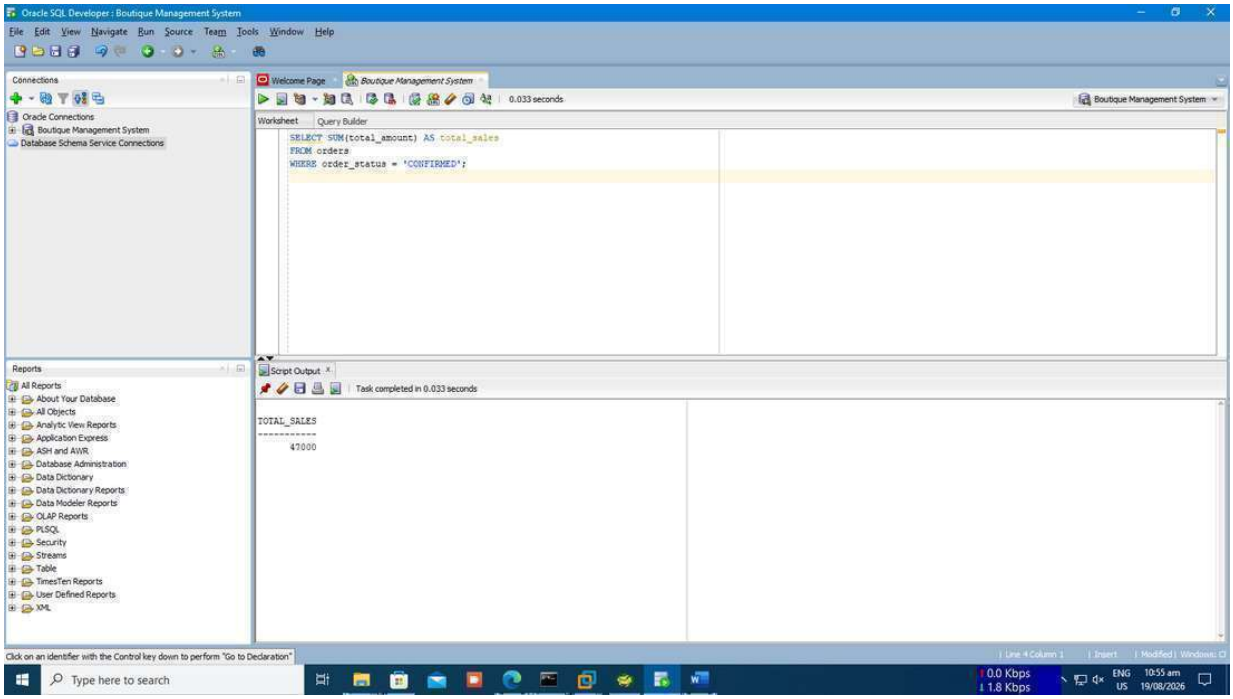
Command

```
SELECT SUM(total_amount) AS total_sales
FROM orders
WHERE order_status = 'CONFIRMED';
```

Explanation

The `SUM()` aggregate function calculates the total amount of confirmed orders.

Figure 6.50: Total Sales



Step 9: Count Total Customers

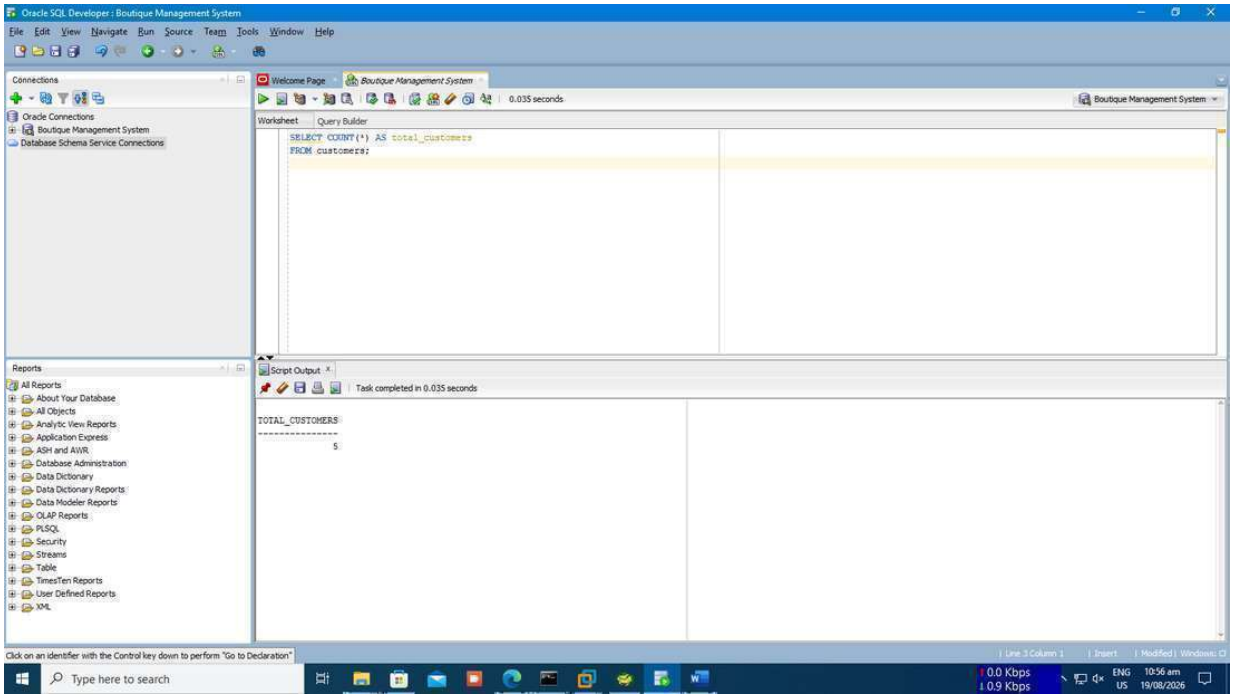
Command

```
SELECT COUNT(*) AS total_customers
FROM customers;
```

Explanation

The `COUNT()` function calculates the total number of customers stored in the database.

Figure 6.51: Total Customer Count



Step 10: Display Complete Order Information

Command

```
SELECT o.order_id,
       c.customer_name,
       e.employee_name,
       p.product_name,
       od.quantity,
       od.subtotal,
       o.total_amount,
       pay.payment_method,
       pay.payment_status
FROM orders o
JOIN customers c
ON o.customer_id = c.customer_id
JOIN employees e
ON o.employee_id = e.employee_id
JOIN order_details od
ON o.order_id = od.order_id
JOIN products p
ON od.product_id = p.product_id
LEFT JOIN payments pay
ON o.order_id = pay.order_id
ORDER BY o.order_id;
```

Explanation

This is one of the most important queries in the project because it combines multiple related tables.

It demonstrates how relational database tables work together to provide meaningful business information.

Figure 6.52: Complete Order Information

